

Pontifícia Universidade Católica de Campinas

Estruturas e Recuperação de Dados II



TAD

PILHAS



Prof. Me. Darcy H.F.R. Moreira



TIPO ABSTRATO DE DADOS- TAD

TIPO ABSTRATO DE DADOS- TAD

Modelo conceitual que organiza e generaliza, especificando:

Conjunto de Dados

Compõem o tipo

```
typedef struct No {  
    int valorNo;  
    struct No* proxNo;  
} No;
```

TIPO ABSTRATO DE DADOS- TAD

Modelo conceitual que organiza e generaliza, especificando:

Conjunto de Dados

Compõem o tipo

```
typedef struct No {  
    int valorNo;  
    struct No* proxNo;  
} No;
```

Operações

Manipulação sobre
sobre os dados

```
Lista* criarLista();  
void inserirInicio(Lista* lista, int valor);  
void inserirFim(Lista* lista, int valor);  
No* buscar(Lista* lista, int valor);  
No *removerNo(Lista* lista, int valor);
```


TIPO ABSTRATO DE DADOS- TAD

Modelo conceitual que organiza e generaliza, especificando:

Conjunto de Dados

Compõem o tipo

```
typedef struct No {  
    int valorNo;  
    struct No* proxNo;  
} No;
```

Operações

Manipulação sobre
sobre os dados

```
Lista* criarLista();  
void inserirInicio(Lista* lista, int valor);  
void inserirFim(Lista* lista, int valor);  
No* buscar(Lista* lista, int valor);  
No *removerNo(Lista* lista, int valor);
```



O que nos interessa: **Comportamento** do tipo (**o que ele faz**) e não na sua implementação interna (**como ele faz**).

TIPO ABSTRATO DE DADOS- TAD

Principais Características

◆ Abstração

O usuário interage apenas com as **operações** definidas, sem conhecer a organização interna dos dados.

TIPO ABSTRATO DE DADOS- TAD

Principais Características

◆ Abstração

O usuário interage apenas com as **operações** definidas, sem conhecer a organização interna dos dados.

OBS: Um **TAD nunca** é acessado diretamente. Todo acesso é feito por meio de **funções e procedimentos**.

TIPO ABSTRATO DE DADOS- TAD

Principais Características

◆ Encapsulamento

Mecanismo que **protege** os dados internos de uma estrutura, permitindo acesso apenas por meio de **funções** definidas.

TIPO ABSTRATO DE DADOS- TAD

Principais Características

◆ Encapsulamento

Mecanismo que **protege** os dados internos de uma estrutura, permitindo acesso apenas por meio de **funções** definidas.

- **Esconder detalhes de implementação**
- **Expor apenas o necessário através de arquivos de cabeçalho (.h):** declara apenas as funções e tipos que outros arquivos precisam conhecer.

TIPO ABSTRATO DE DADOS- TAD

Principais Características

- ◆ **Separação Interface × Implementação (modularização)**

TIPO ABSTRATO DE DADOS- TAD

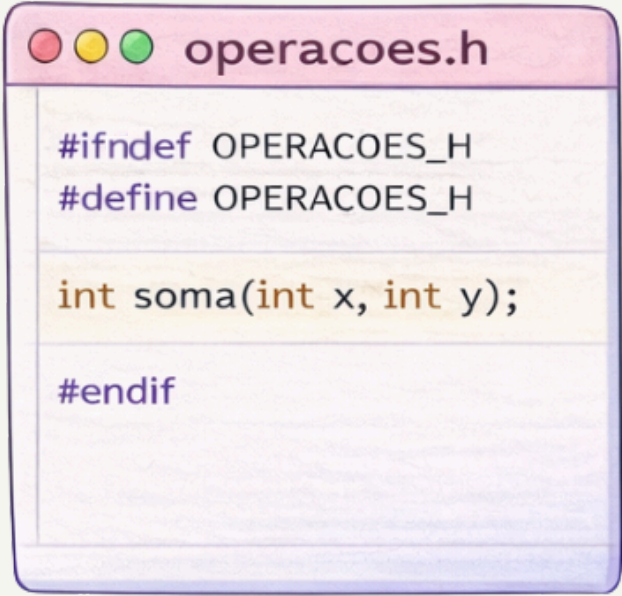
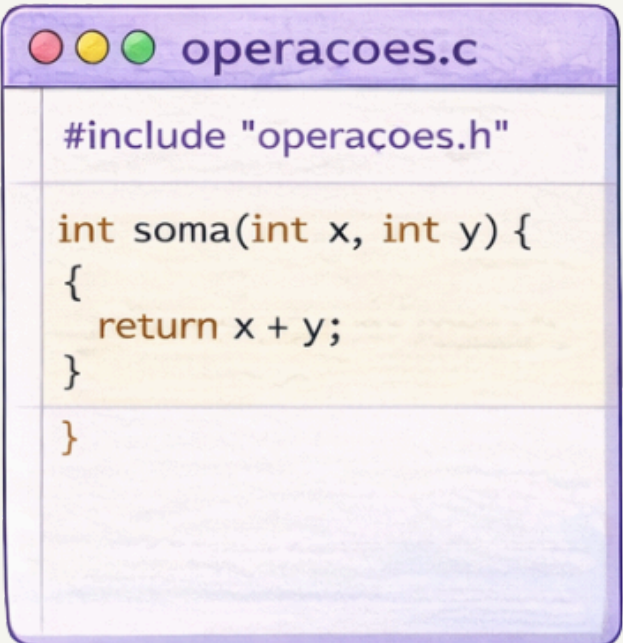
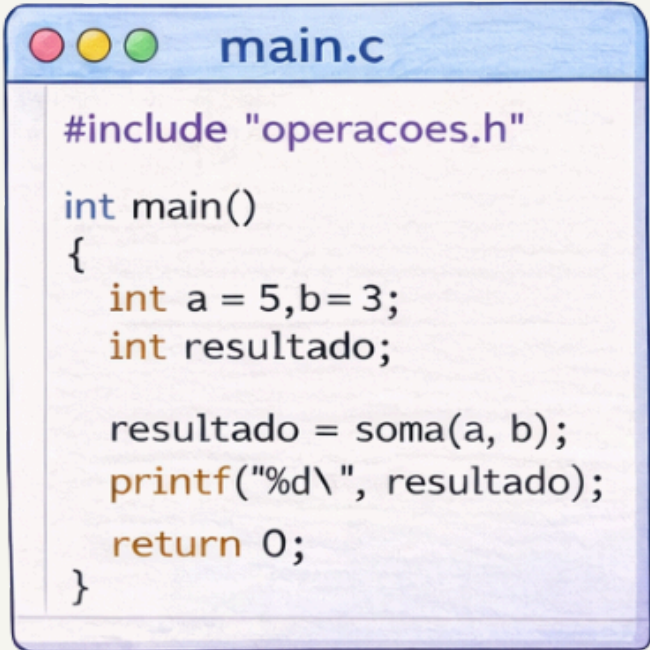
Principais Características

◆ Separação Interface × Implementação (modularização)

Em C:

 **.h** → **interface** (o que pode ser usado)

 **.c** → **implementação** (como funciona internamente)

 <pre> #ifndef OPERACOES_H #define OPERACOES_H int soma(int x, int y); #endif </pre>	 <pre> #include "operacoes.h" int soma(int x, int y) { return x + y; } </pre>	 <pre> #include "operacoes.h" int main() { int a = 5, b = 3; int resultado; resultado = soma(a, b); printf("%d\\", resultado); return 0; } </pre>
Declarações	Implementações	Chamada das funções

TIPO ABSTRATO DE DADOS- TAD

Essa **separação** permite modificar a **implementação(.c)** sem impactar o código que utiliza o **TAD (.h)**

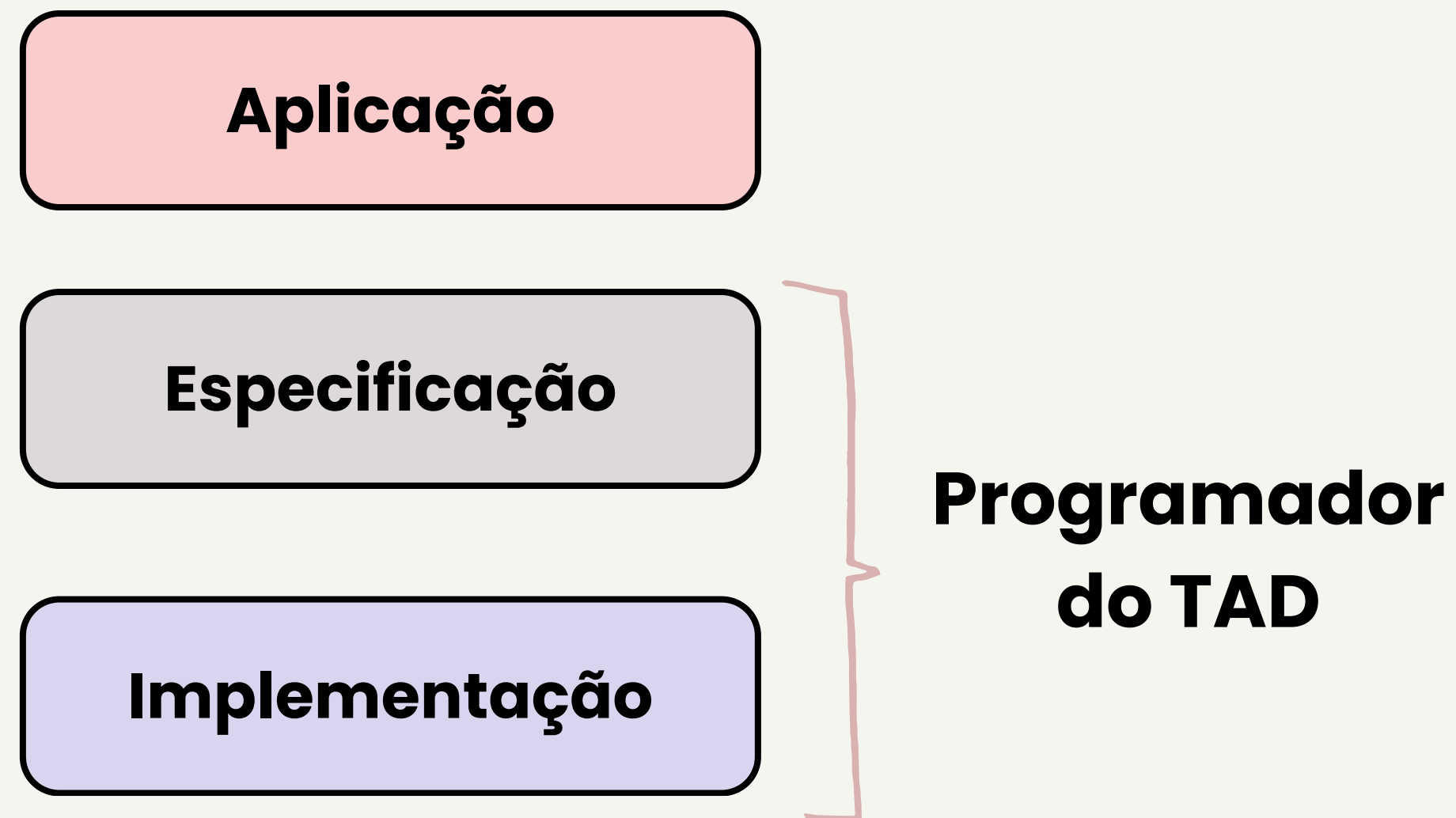
Aplicação

Especificação

Implementação

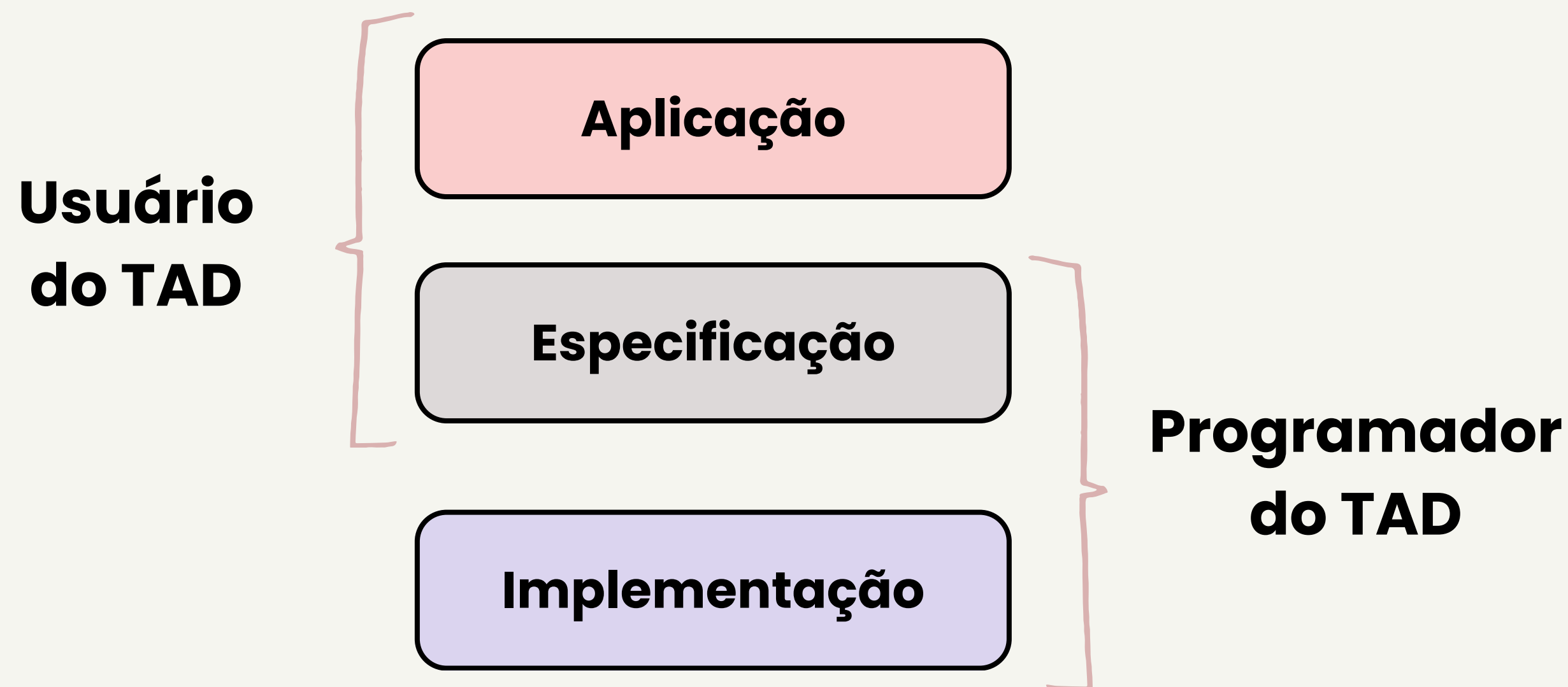
TIPO ABSTRATO DE DADOS- TAD

Essa **separação** permite modificar a **implementação(.c)** sem impactar o código que utiliza o **TAD (.h)**



TIPO ABSTRATO DE DADOS- TAD

Essa **separação** permite modificar a **implementação(.c)** sem impactar o código que utiliza o **TAD (.h)**



TIPO ABSTRATO DE DADOS- TAD

Vantagens do Uso de **TAD**

- Maior **portabilidade**
- Facilidade de **manutenção**
- Melhor **organização** do código
- Aumento da **segurança**
- Maior **clareza** na modelagem de algoritmos

TIPO ABSTRATO DE DADOS- TAD

Para executar, primeiro preciso compilar todos os arquivos-fonte (.c) associados ao projeto e gerar o executável.

```
gcc main.c lista.c -o programa
```

TIPO ABSTRATO DE DADOS- TAD

Para executar, primeiro preciso compilar todos os arquivos-fonte (.c) associados ao projeto e gerar o executável.

```
gcc main.c lista.c -o programa
```

Ou compilando separadamente:

1. Compilando o **arquivo-fonte (.c)** e gerando um **arquivo objeto (.o)**:

```
gcc -c lista.c -o lista.o  
gcc -c main.c -o main.o
```


TIPO ABSTRATO DE DADOS- TAD

- Para executar, primeiro preciso compilar todos os arquivos-fonte (.c) associados ao projeto e gerar o executável.

```
gcc main.c lista.c -o programa
```

- Ou compilando separadamente:

1. Compilando o **arquivo-fonte (.c)** e gerando um **arquivo objeto (.o)**:

```
gcc -c lista.c -o lista.o  
gcc -c main.c -o main.o
```

2. Linkando os **arquivos objetos** para criar o **executável**

```
gcc lista.o main.o -o programa
```

Assim, você só precisa recompilar o arquivo se ele for modificado, sem necessidade de compilar tudo toda vez.

TIPO ABSTRATO DE DADOS- TAD

Alternativa: Usar um Makefile para automatizar o processo (projetos com múltiplos arquivos).

```
# Nome do executável
EXEC = programa

# Arquivos fonte
SRCS = main.c lista.c

# Arquivos objeto
OBJS = $(SRCS:.c=.o)

# Compilador
CC = gcc

# Flags do compilador
CFLAGS = -Wall -g

# Regra padrão
all: $(EXEC)

# Regra para criar o executável
$(EXEC): $(OBJS)
    $(CC) $(OBJS) -o $@

# Regra para compilar arquivos .c em .o
%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

# Limpar arquivos gerados
clean:
    rm -f $(OBJS) $(EXEC)
```



Você só precisa rodar **make** para compilar o programa.
Ele **recompilará** apenas os arquivos que foram **modificados**.

PILHAS

- Baseada no princípio **LIFO** (**Last In, First Out**).
- Elementos (nós da pilha) são inseridos e removidos do **topo**.



PILHAS

- Baseada no princípio **LIFO** (**Last In, First Out**).
- Elementos (nós da pilha) são inseridos e removidos do **topo**.

Características

- **Lineares**: Organizados em sequência.
- **Acesso restrito**: Somente é possível acessar o topo da pilha
- **Mais flexíveis**: podem ser estáticas ou dinâmicas



PILHAS

- Baseada no princípio **LIFO** (**Last In, First Out**).
- Elementos (nós da pilha) são inseridos e removidos do **topo**.

Características

- **Lineares**: Organizados em sequência.
- **Acesso restrito**: Somente é possível acessar o topo da pilha
- **Mais flexíveis**: podem ser estáticas ou dinâmicas



Operações básicas



Criar()

PILHAS

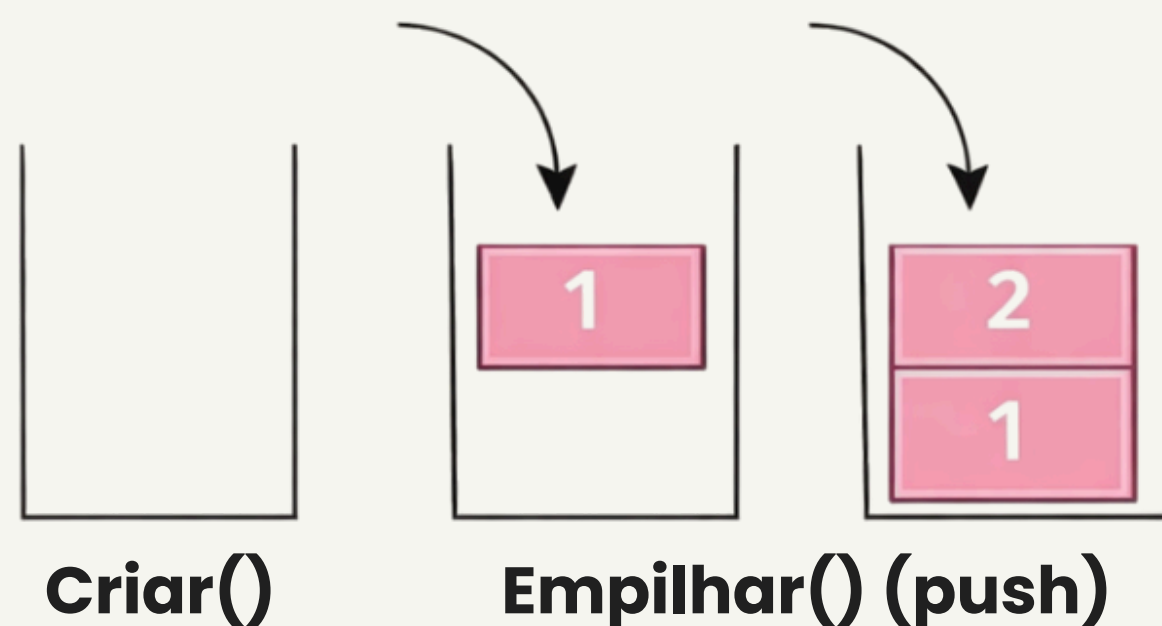
- Baseada no princípio **LIFO** (**Last In, First Out**).
- Elementos (nós da pilha) são inseridos e removidos do **topo**.

Características

- **Lineares**: Organizados em sequência.
- **Acesso restrito**: Somente é possível acessar o topo da pilha
- **Mais flexíveis**: podem ser estáticas ou dinâmicas



Operações básicas



PILHAS

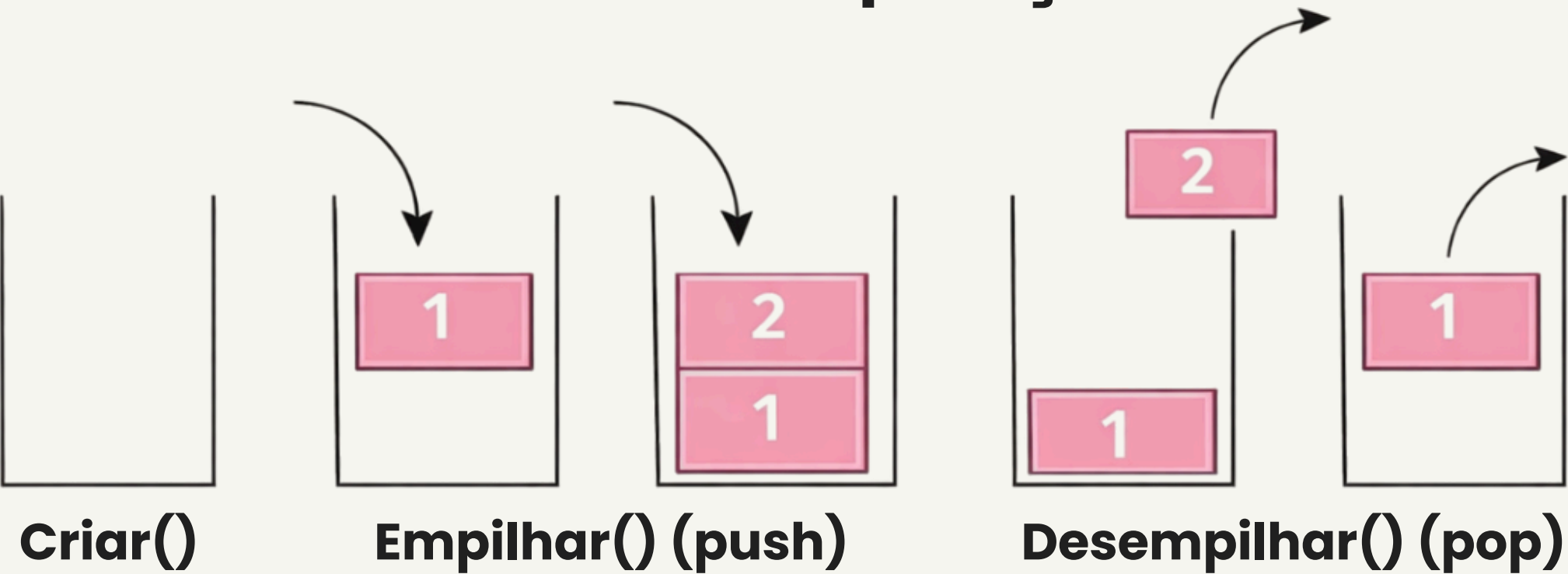
- Baseada no princípio **LIFO** (**Last In, First Out**).
- Elementos (nós da pilha) são inseridos e removidos do **topo**.



Características

- **Lineares**: Organizados em sequência.
- **Acesso restrito**: Somente é possível acessar o topo da pilha
- **Mais flexíveis**: podem ser estáticas ou dinâmicas

Operações básicas



PILHAS

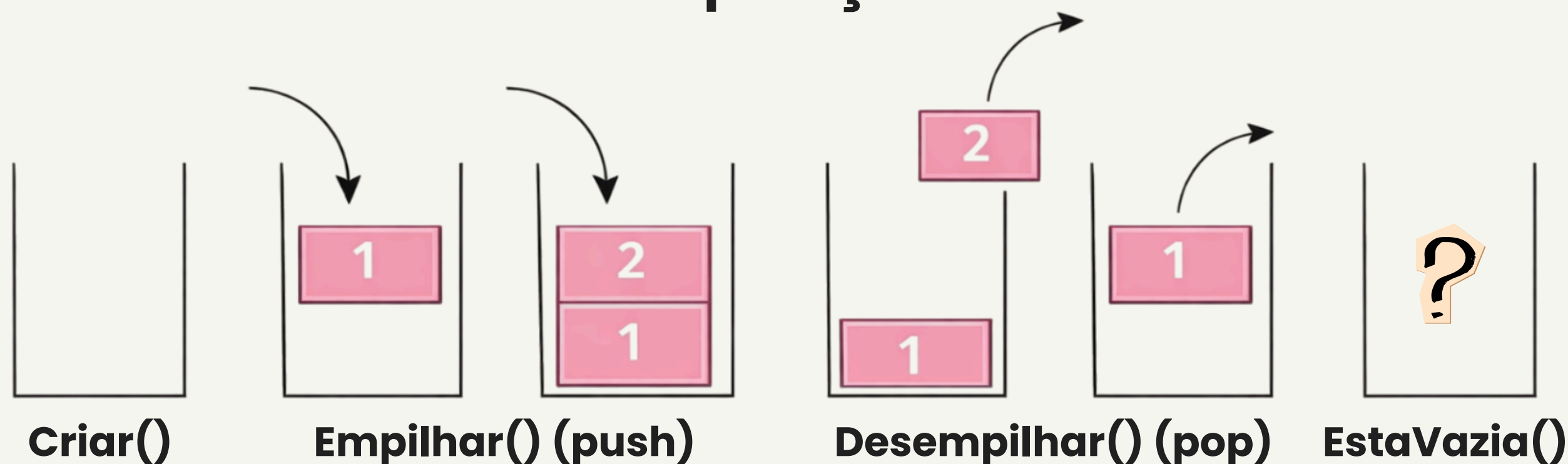
- Baseada no princípio **LIFO** (**Last In, First Out**).
- Elementos (nós da pilha) são inseridos e removidos do **topo**.

Características

- **Lineares**: Organizados em sequência.
- **Acesso restrito**: Somente é possível acessar o topo da pilha
- **Mais flexíveis**: podem ser estáticas ou dinâmicas



Operações básicas



PILHAS

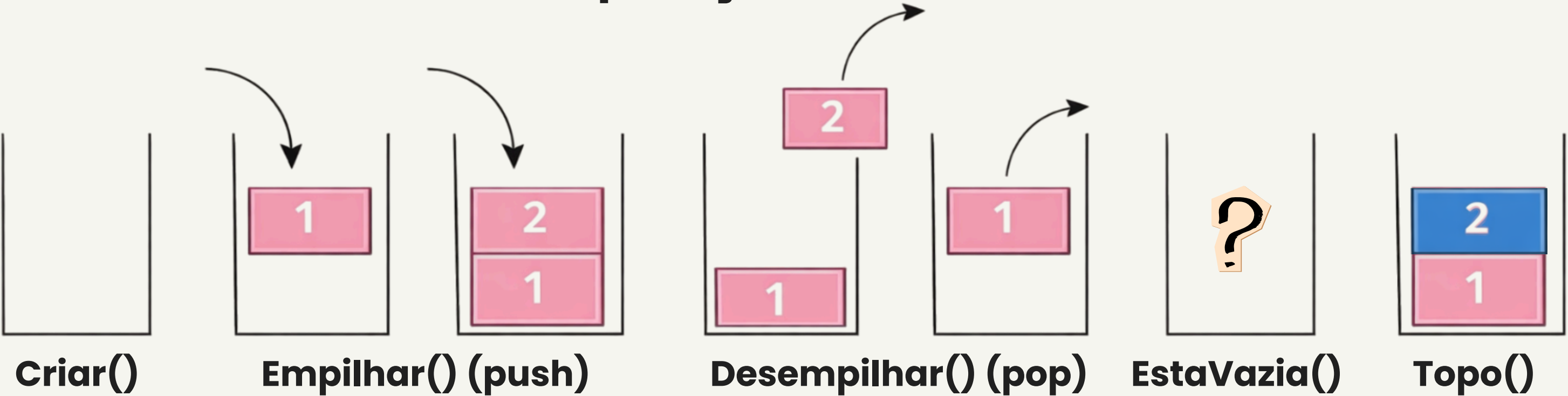
- Baseada no princípio **LIFO** (**Last In, First Out**).
- Elementos (nós da pilha) são inseridos e removidos do **topo**.



Características

- **Lineares**: Organizados em sequência.
- **Acesso restrito**: Somente é possível acessar o topo da pilha
- **Mais flexíveis**: podem ser estáticas ou dinâmicas

Operações básicas



PILHAS



- Implementação **simples**
- Ideal para problemas de **desfazer/refazer** ações
- Útil para gerenciar processos **recursivos**

PILHAS



- Implementação **simples**
- Ideal para problemas de **desfazer/refazer** ações
- Útil para gerenciar processos **recursivos**



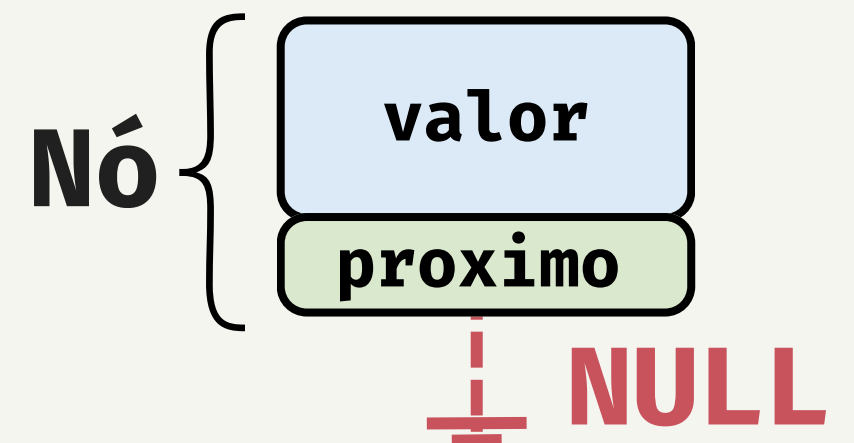
- Acesso **restrito**: só é possível manipular o **topo**

PILHAS

- Montagem como uma **lista simplesmente encadeada**

Estrutura base

```
typedef struct NoPilha{  
    int valor;  
    struct NoPilha* prox;  
} NoPilha;
```



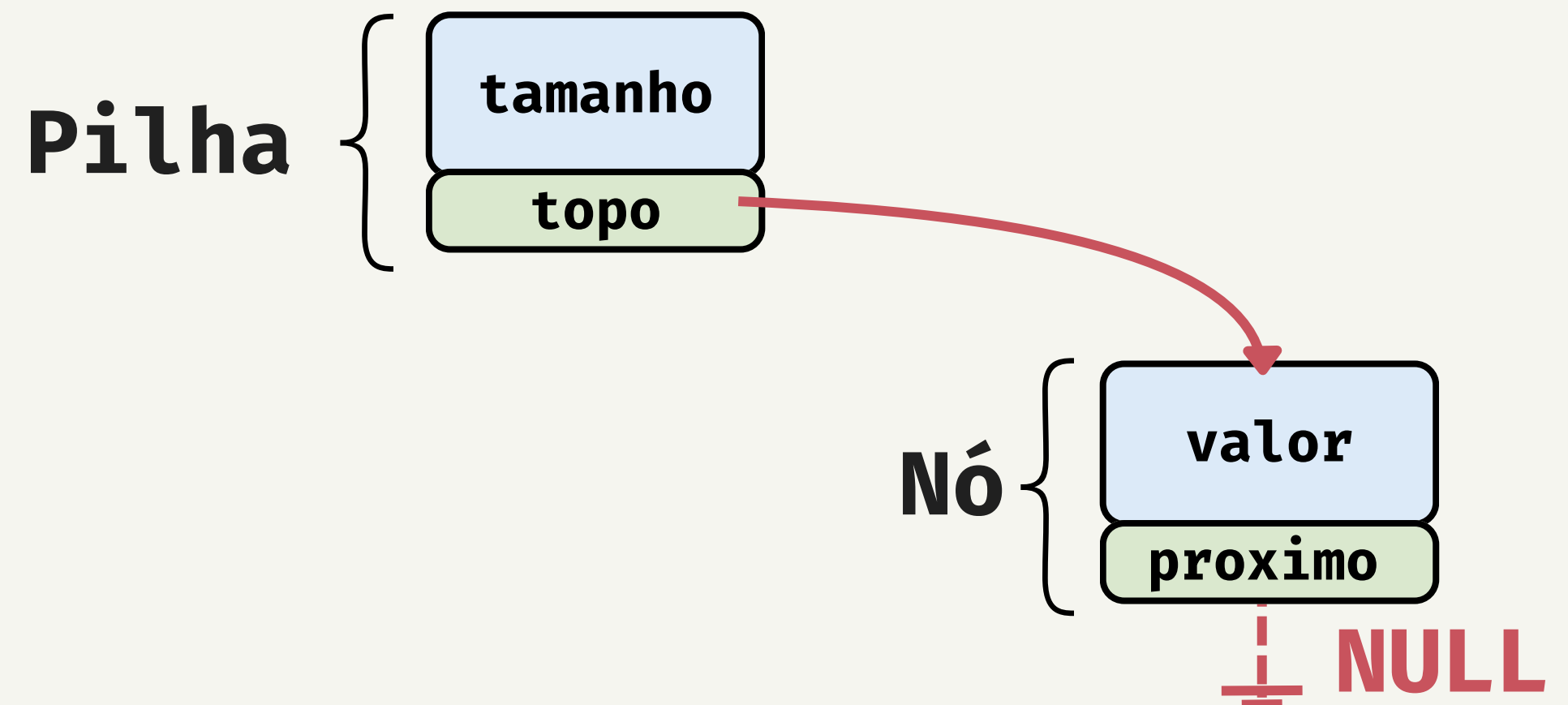
PILHAS

- Montagem como uma **lista simplesmente encadeada**

Estrutura base

```
typedef struct NoPilha{
    int valor;
    struct NoPilha* prox;
} NoPilha;
```

```
typedef struct Pilha{
    NoPilha* topo;
    int tamanho;
} Pilha;
```



PILHAS

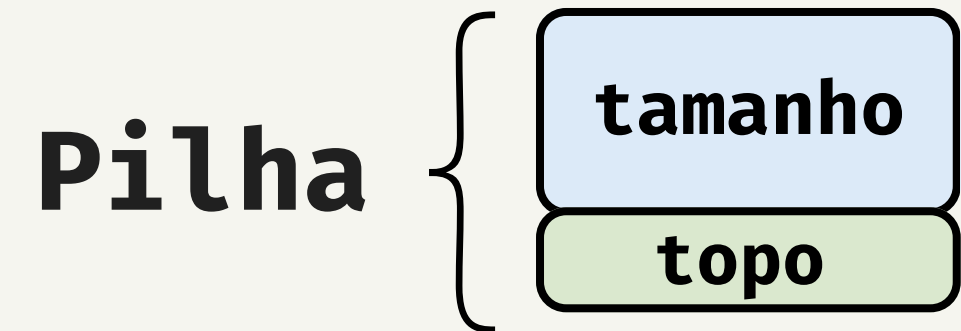
Criando uma struct Pilha

```
Pilha *criarPilha() {  
    Pilha *novaPilha = (Pilha*)malloc(sizeof(Pilha));  
    if (!novaPilha) {  
        printf("Erro ao alocar memória para a pilha.\n");  
    }  
    novaPilha->topo = NULL;  
    novaPilha->tamanho = 0;  
    return novaPilha;  
}
```

PILHAS

Criando uma struct Pilha

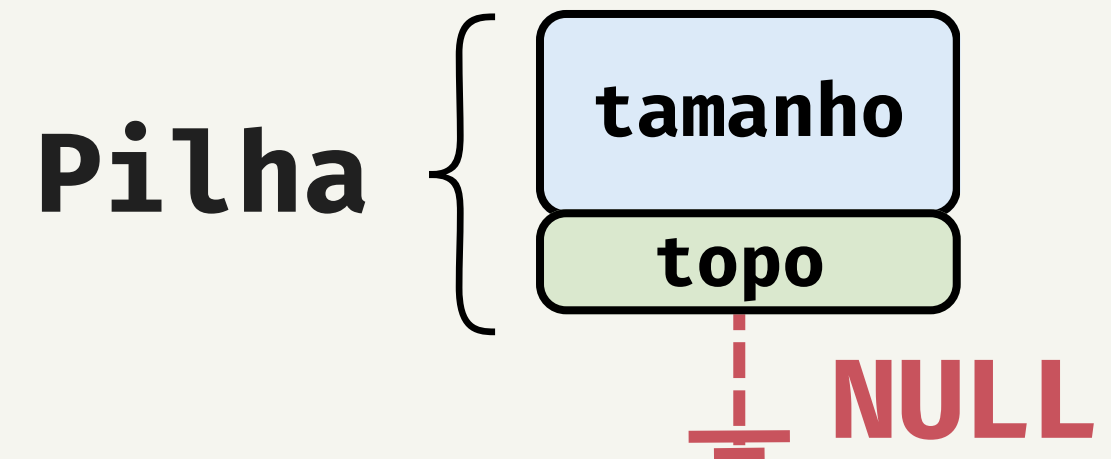
```
Pilha *criarPilha() {  
    Pilha *novaPilha = (Pilha*)malloc(sizeof(Pilha));  
    if (!novaPilha) {  
        printf("Erro ao alocar memória para a pilha.\n");  
    }  
    novaPilha->topo = NULL;  
    novaPilha->tamanho = 0;  
    return novaPilha;  
}
```



PILHAS

Criando uma struct Pilha

```
Pilha *criarPilha() {  
    Pilha *novaPilha = (Pilha*)malloc(sizeof(Pilha));  
    if (!novaPilha) {  
        printf("Erro ao alocar memória para a pilha.\n");  
    }  
    novaPilha->topo = NULL;  
    novaPilha->tamanho = 0;  
    return novaPilha;  
}
```



PILHAS

Criando uma struct Pilha

Pilha *criarPilha() {

Pilha *novaPilha = (Pilha*)malloc(sizeof(Pilha));

if (!novaPilha) {

printf("Erro ao alocar memória para a pilha.\n");

}

novaPilha->topo = NULL;

novaPilha->tamanho = 0;

return novaPilha;

}

Pilha

0

topo

⌞ NULL

34

PILHAS

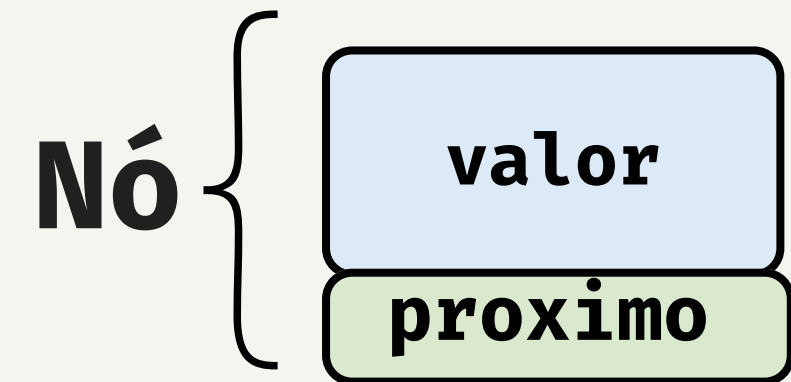
Criando uma struct NoPilha

```
NoPilha* criarNo(int valor) {  
    NoPilha* novoNo = (NoPilha*)malloc(sizeof(NoPilha));  
    if (!novoNo) {  
        printf("Erro ao alocar memória para o nó da pilha.\n");  
    }  
    novoNo->valor = valor;  
    novoNo->prox = NULL;  
    return novoNo;  
}
```

PILHAS

Criando uma struct NoPilha

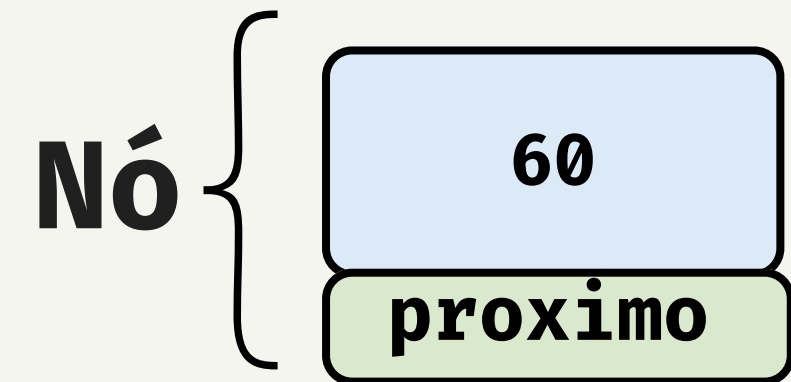
```
NoPilha* criarNo(int valor) {  
    NoPilha* novoNo = (NoPilha*)malloc(sizeof(NoPilha));  
    if (!novoNo) {  
        printf("Erro ao alocar memória para o nó da pilha.\n");  
    }  
    novoNo->valor = valor;  
    novoNo->prox = NULL;  
    return novoNo;  
}
```



PILHAS

Criando uma struct NoPilha

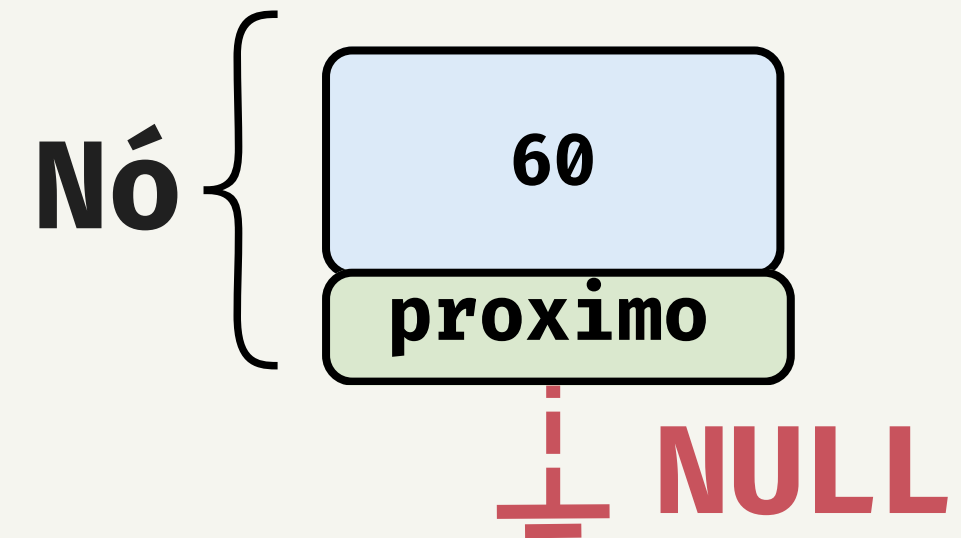
```
NoPilha* criarNo(int valor) {  
    NoPilha* novoNo = (NoPilha*)malloc(sizeof(NoPilha));  
    if (!novoNo) {  
        printf("Erro ao alocar memória para o nó da pilha.\n");  
    }  
    novoNo->valor = valor;  
    novoNo->prox = NULL;  
    return novoNo;  
}
```



PILHAS

Criando uma struct NoPilha

```
NoPilha* criarNo(int valor) {  
    NoPilha* novoNo = (NoPilha*)malloc(sizeof(NoPilha));  
    if (!novoNo) {  
        printf("Erro ao alocar memória para o nó da pilha.\n");  
    }  
    novoNo->valor = valor;  
    novoNo->prox = NULL;  
    return novoNo;  
}
```

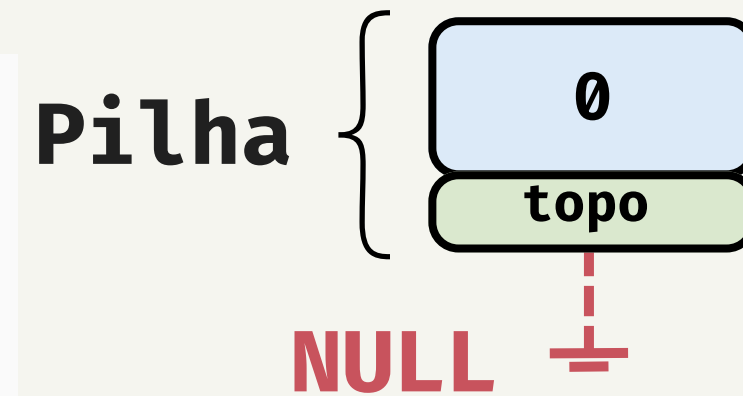


PILHAS

Empilhar: Inserir no topo

Caso 1: Pilha vazia

```
void empilhar(Pilha* pilha, int valor) {  
    NoPilha* novoNo = criarNo(valor);  
    novoNo->prox = pilha->topo;  
    pilha->topo = novoNo;  
    pilha->tamanho++;  
}
```

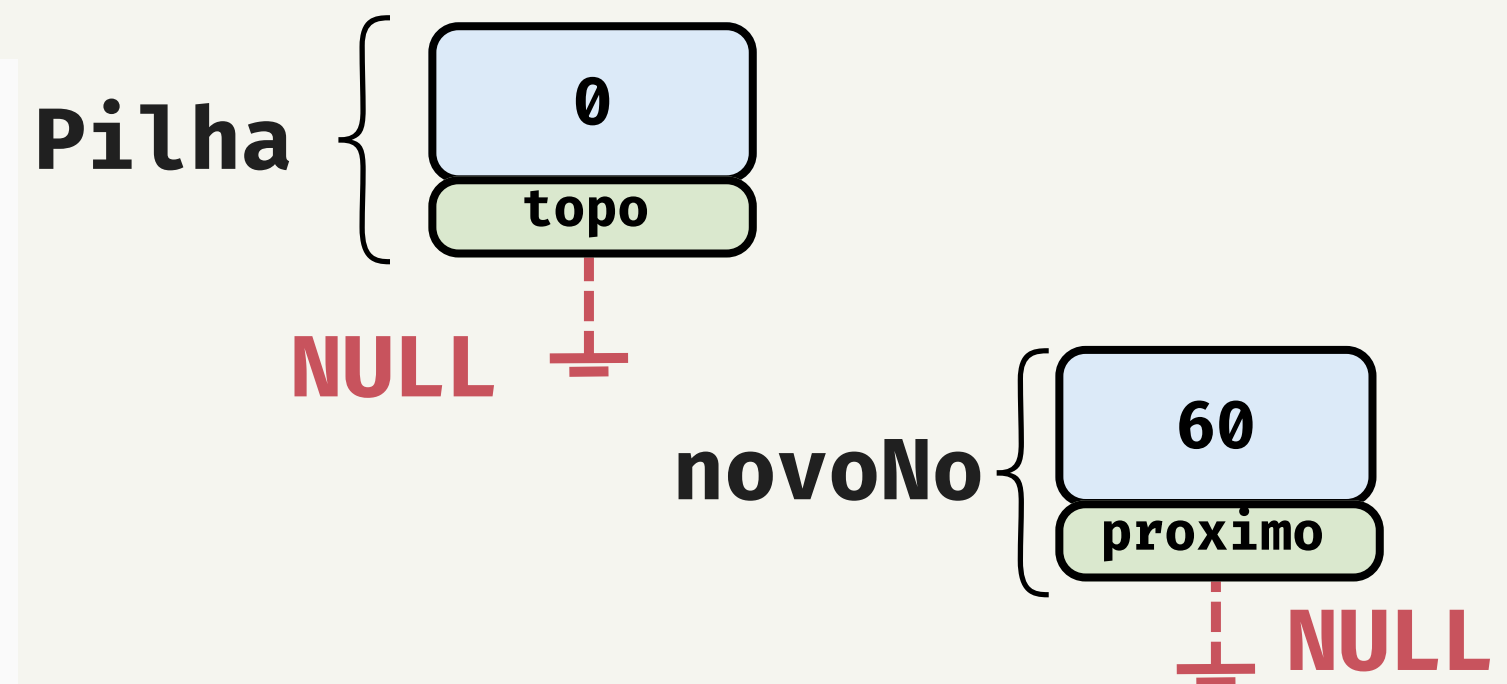


PILHAS

Empilhar: Inserir no topo

Caso 1: Pilha vazia

```
void empilhar(Pilha* pilha, int valor) {
    NoPilha* novoNo = criarNo(valor);
    novoNo->prox = pilha->topo;
    pilha->topo = novoNo;
    pilha->tamanho++;
}
```

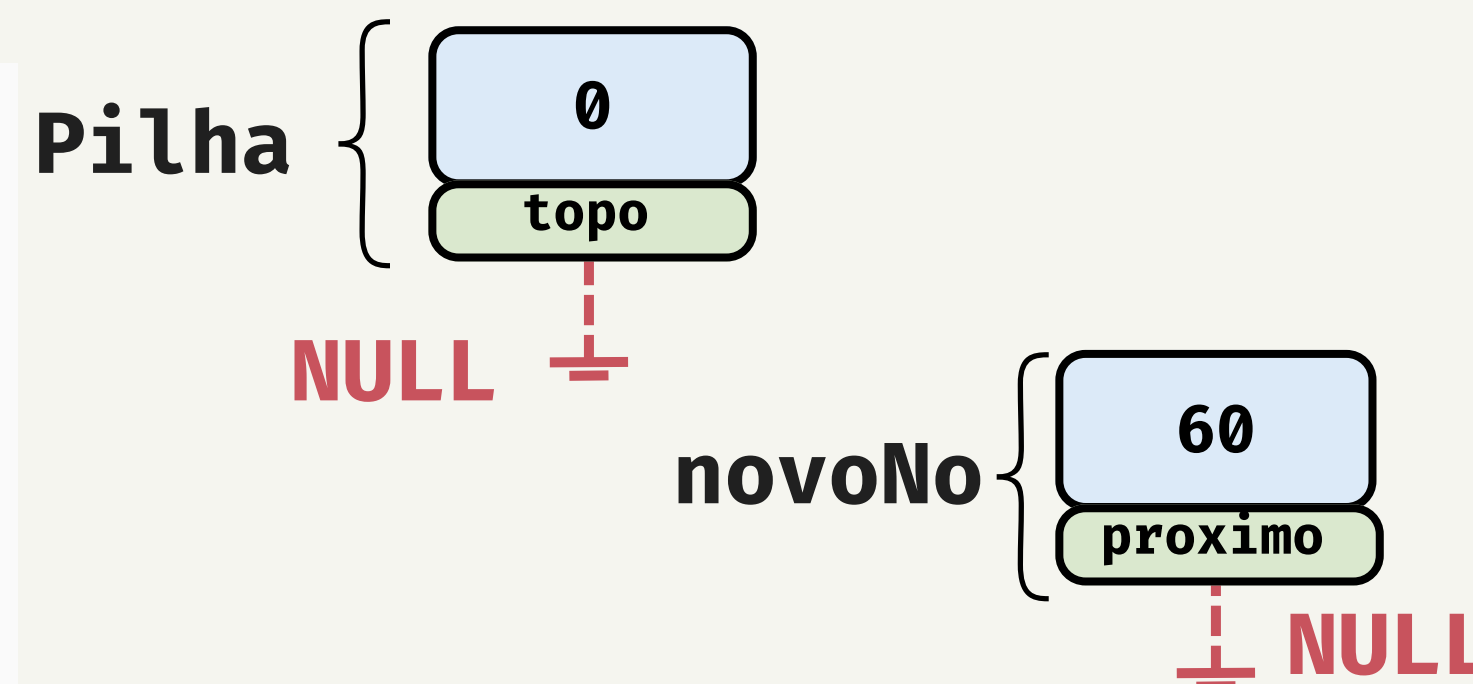


PILHAS

Empilhar: Inserir no topo

Caso 1: Pilha vazia

```
void empilhar(Pilha* pilha, int valor) {
    NoPilha* novoNo = criarNo(valor);
    novoNo->prox = pilha->topo;
    pilha->topo = novoNo;
    pilha->tamanho++;
}
```

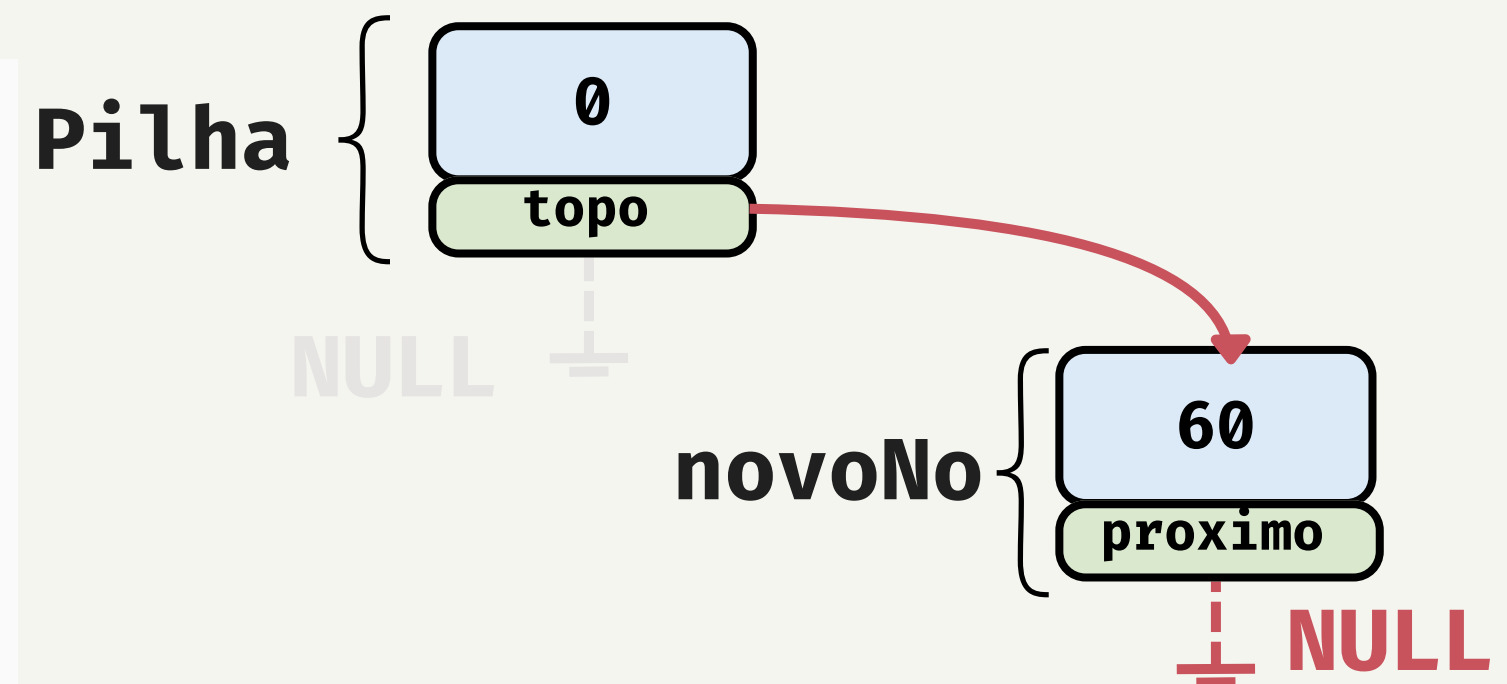


PILHAS

Empilhar: Inserir no topo

Caso 1: Pilha vazia

```
void empilhar(Pilha* pilha, int valor) {
    NoPilha* novoNo = criarNo(valor);
    novoNo->prox = pilha->topo;
    pilha->topo = novoNo;
    pilha->tamanho++;
}
```



PILHAS

Empilhar: Inserir no topo

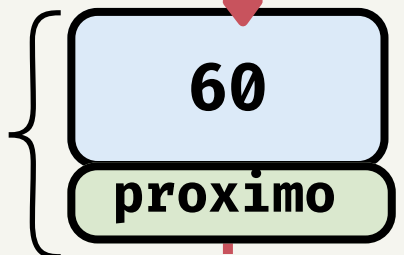
Caso 1: Pilha vazia

```
void empilhar(Pilha* pilha, int valor) {  
    NoPilha* novoNo = criarNo(valor);  
    novoNo->prox = pilha->topo;  
    pilha->topo = novoNo;  
    pilha->tamanho++;  
}
```

Pilha



novoNo



⊥ NULL

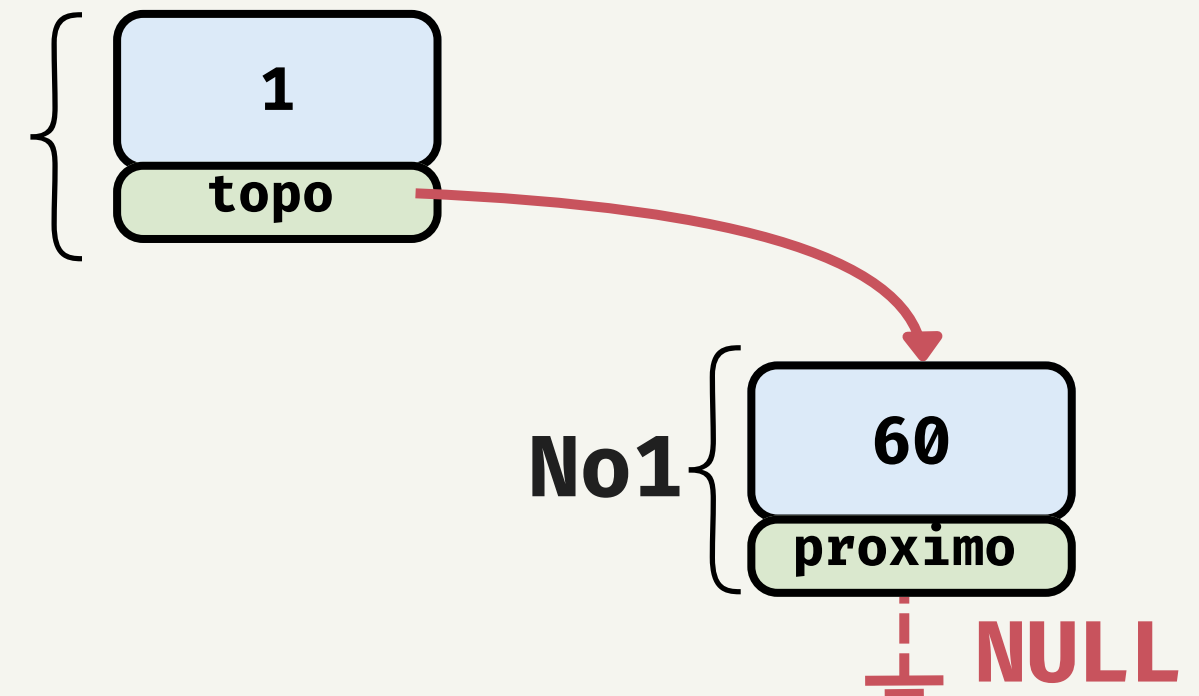
PILHAS

Empilhar: Inserir no topo

Caso 2: Pilha com elementos

```
void empilhar(Pilha* pilha, int valor) {
    NoPilha* novoNo = criarNo(valor);
    novoNo->prox = pilha->topo;
    pilha->topo = novoNo;
    pilha->tamanho++;
}
```

Pilha



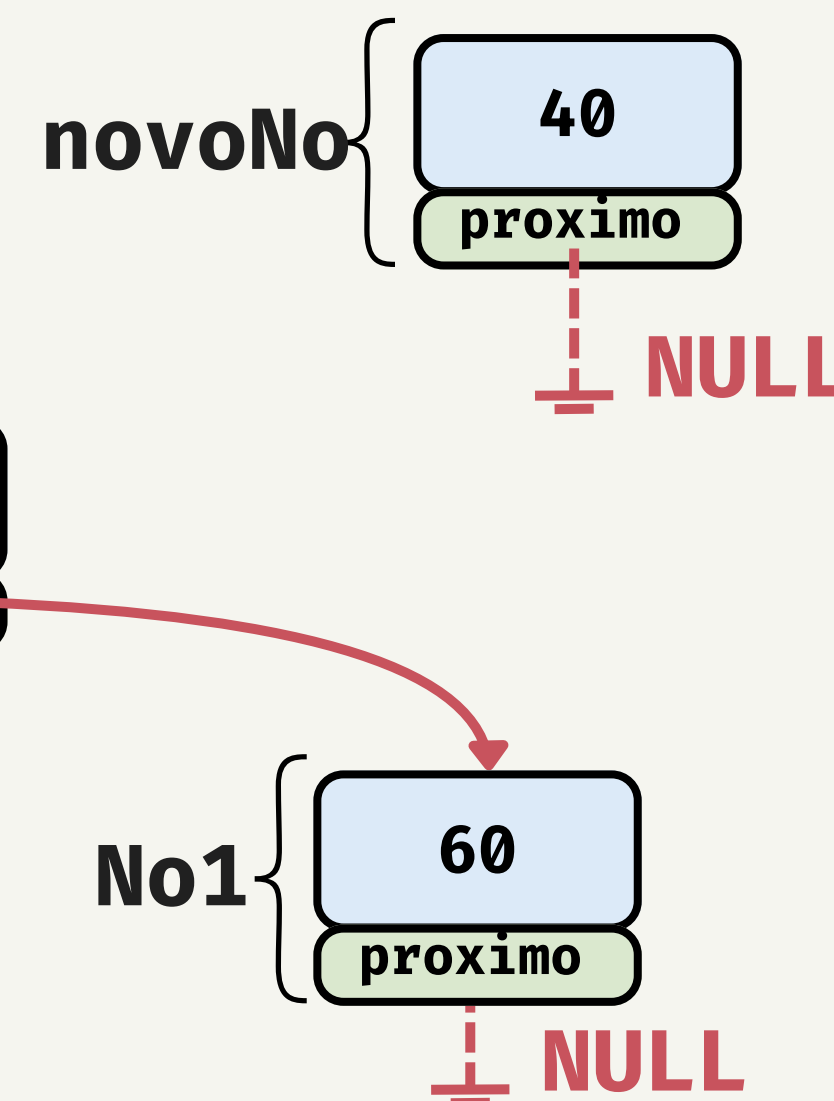
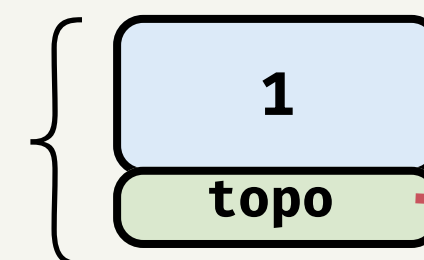
PILHAS

Empilhar: Inserir no topo

Caso 2: Pilha com um elemento

```
void empilhar(Pilha* pilha, int valor) {
    NoPilha* novoNo = criarNo(valor);
    novoNo->prox = pilha->topo;
    pilha->topo = novoNo;
    pilha->tamanho++;
}
```

Pilha

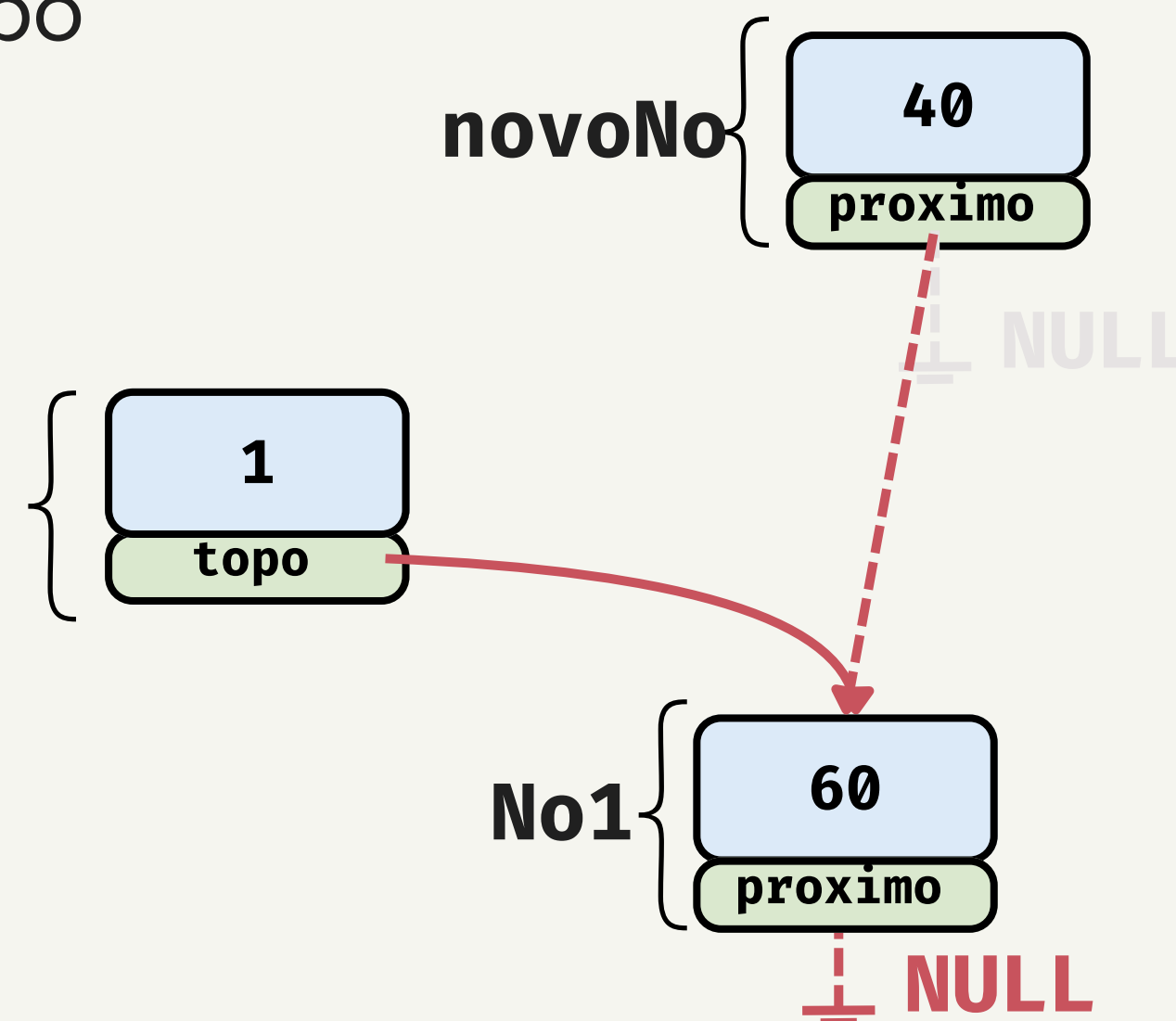


PILHAS

Empilhar: Inserir no topo

```
void empilhar(Pilha* pilha, int valor) {
    NoPilha* novoNo = criarNo(valor);
    novoNo->prox = pilha->topo;
    pilha->topo = novoNo;
    pilha->tamanho++;
}
```

Pilha

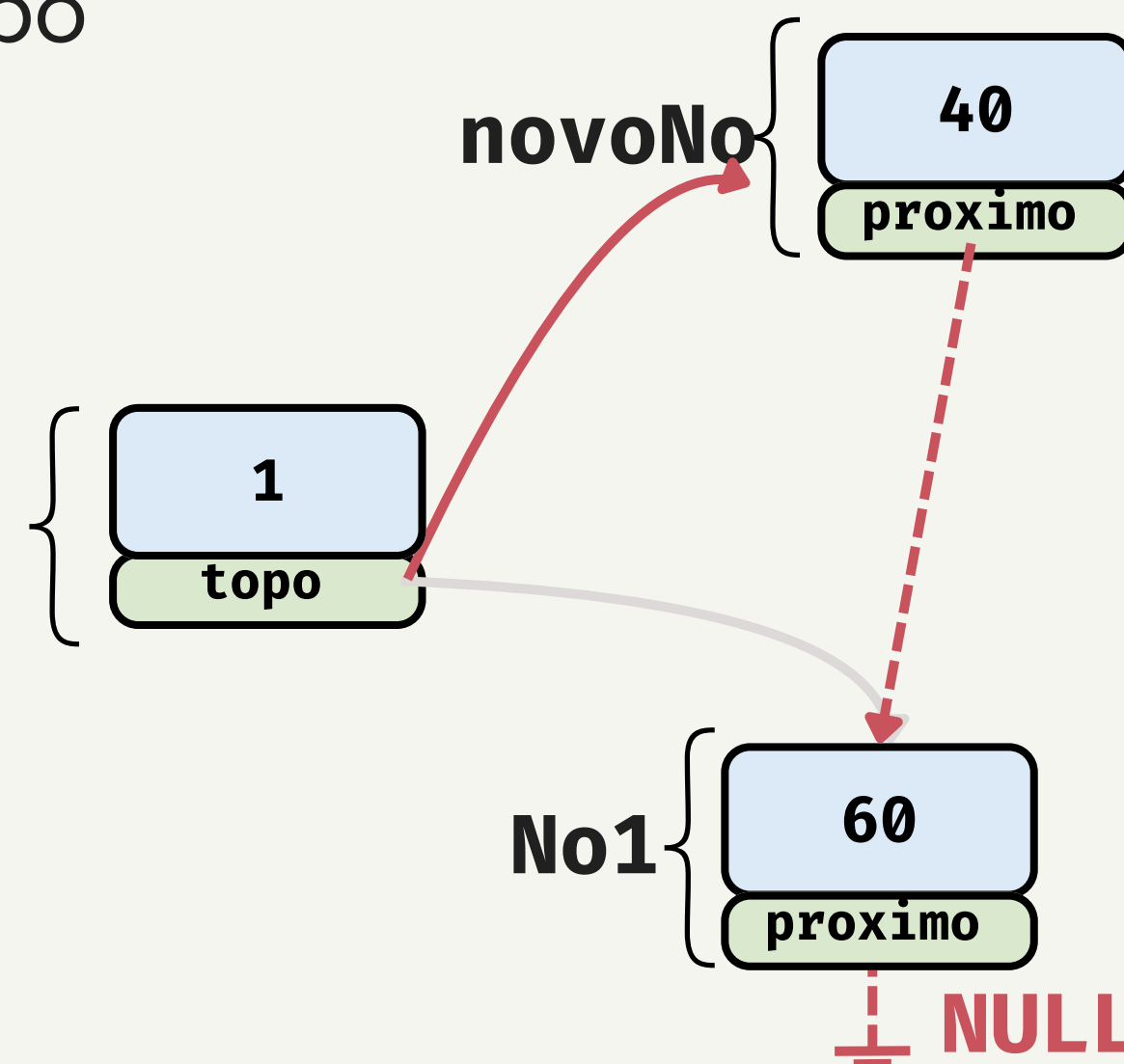


PILHAS

Empilhar: Inserir no topo

```
void empilhar(Pilha* pilha, int valor) {
    NoPilha* novoNo = criarNo(valor);
    novoNo->prox = pilha->topo;
    pilha->topo = novoNo;
    pilha->tamanho++;
}
```

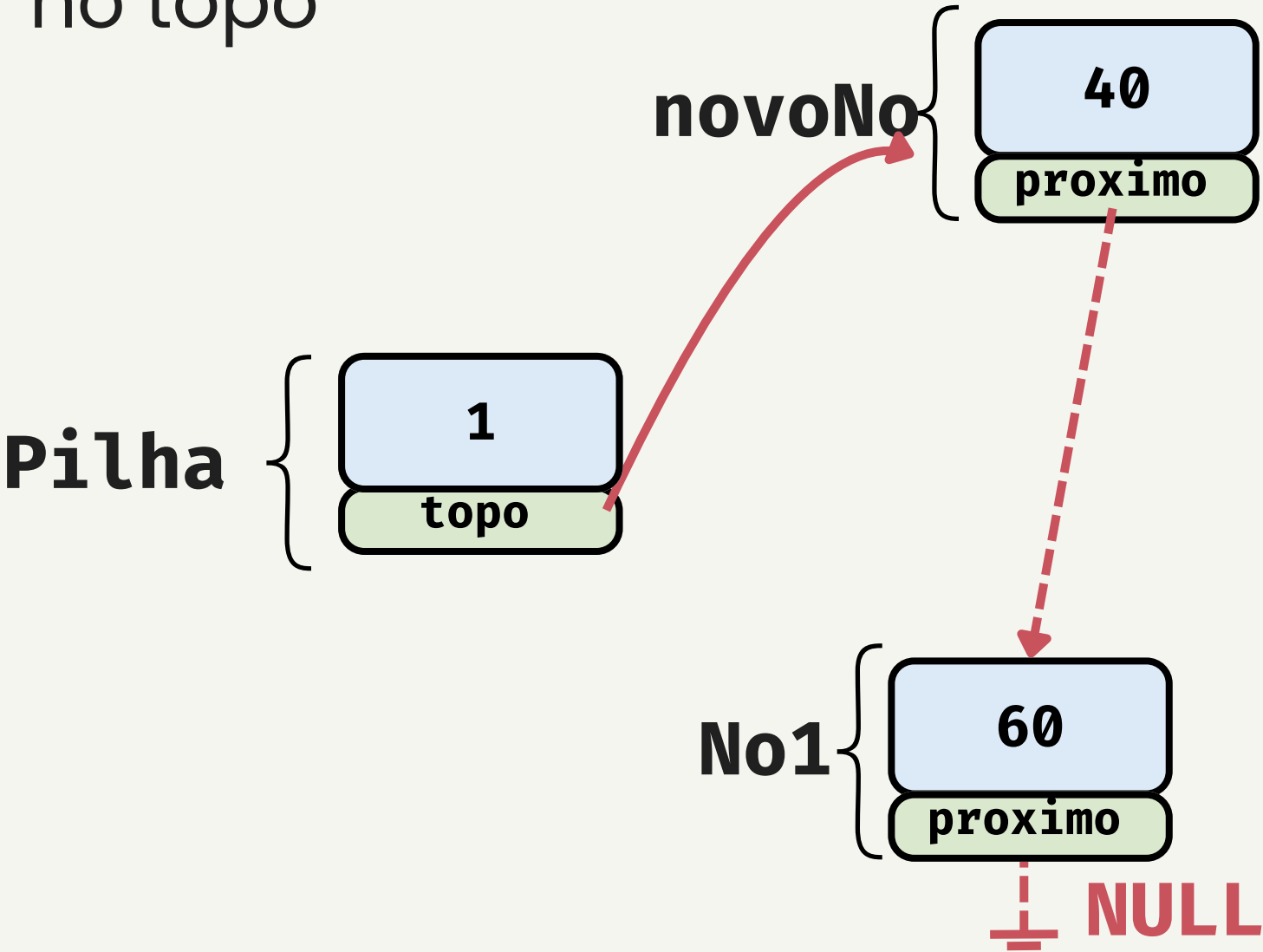
Pilha



PILHAS

Empilhar: Inserir no topo

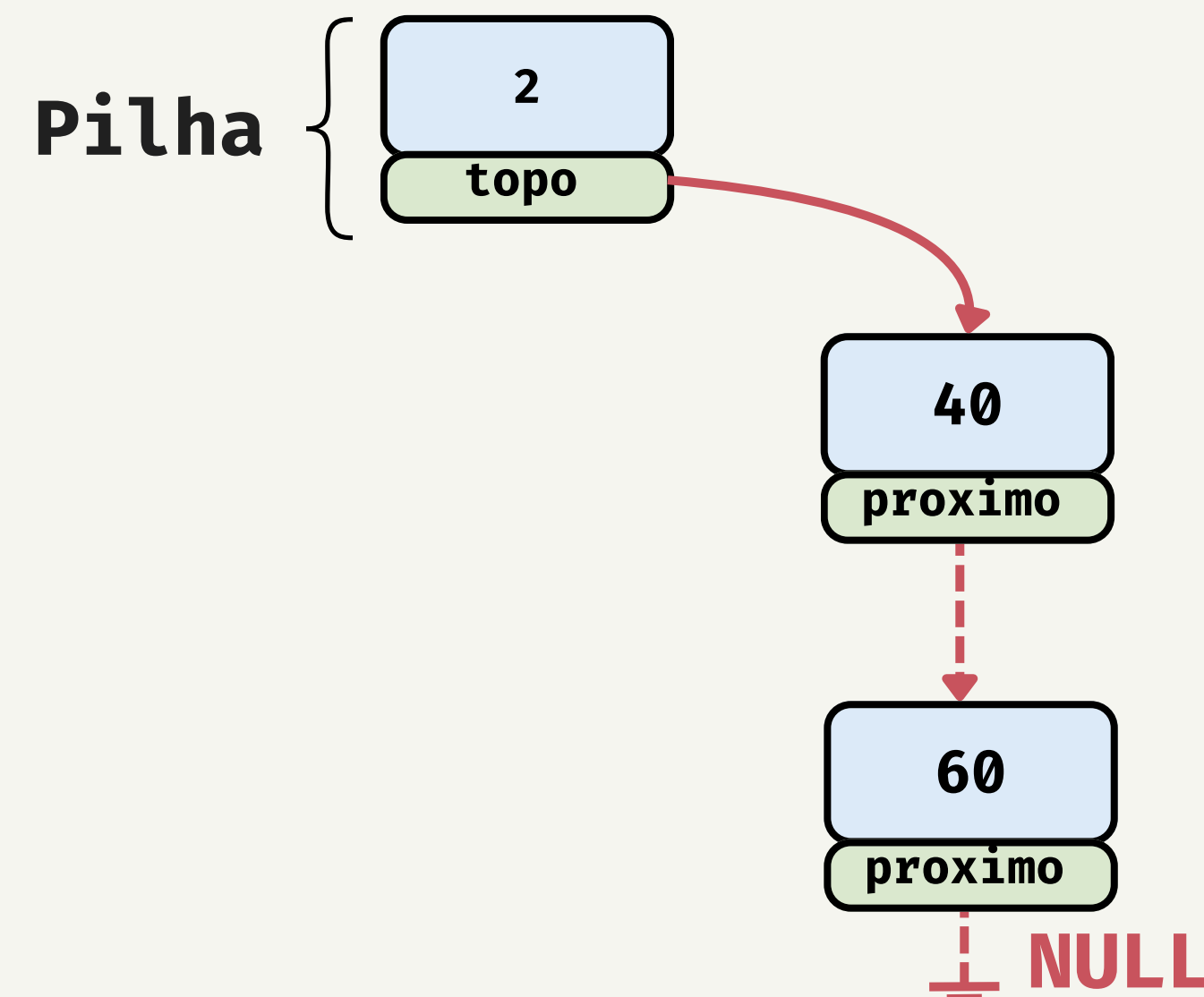
```
void empilhar(Pilha* pilha, int valor) {
    NoPilha* novoNo = criarNo(valor);
    novoNo->prox = pilha->topo;
    pilha->topo = novoNo;
    pilha->tamanho++;
}
```



PILHAS

Desempilhar: Remover do topo

```
void desempilhar(Pilha* pilha, int* valorDesempilhado) {
    if (pilhaVazia(pilha)) {
        printf("Pilha vazia. Não é possível desempilhar.\n");
        return;
    }
    NoPilha* temp = pilha->topo;
    *valorDesempilhado = temp->valor;
    pilha->topo = temp->prox;
    free(temp);
    pilha->tamanho--;
}
```

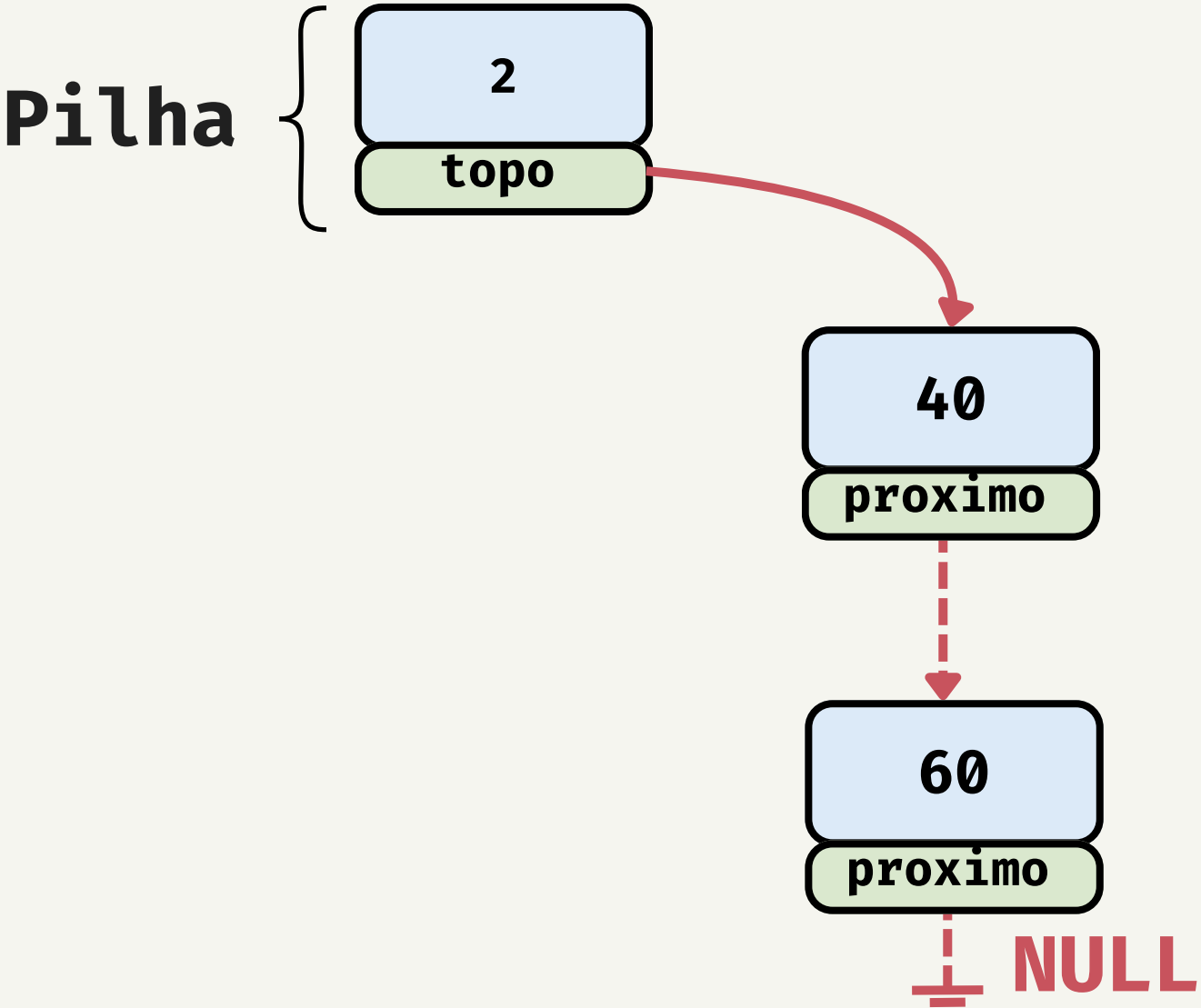


O **valorDesempilhado** será declarado fora e recebe o valor que está no nó desempilhado

PILHAS

Desempilhar: Remover do topo

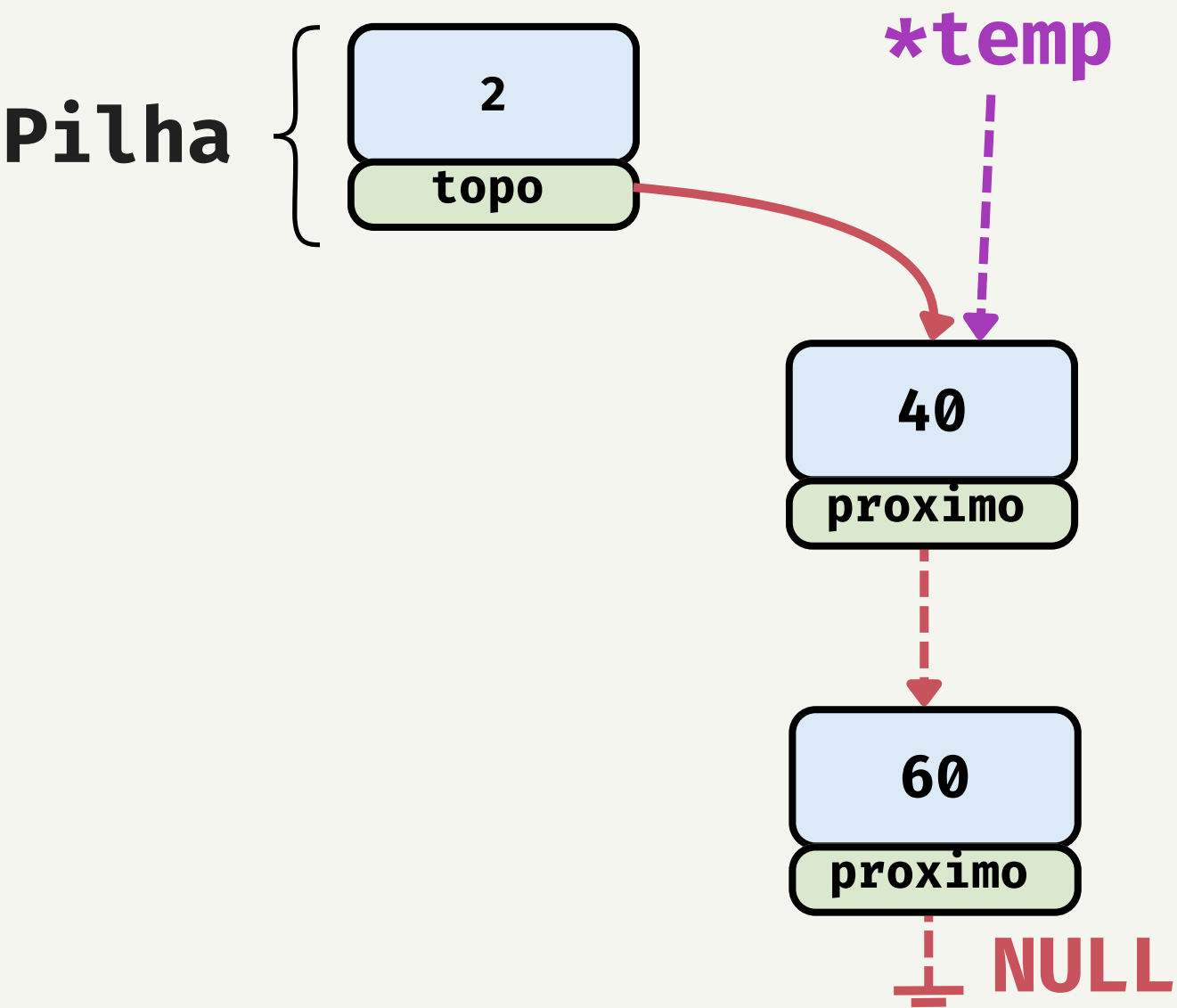
```
void desempilhar(Pilha* pilha, int* valorDesempilhado) {
    if (pilhaVazia(pilha)) {
        printf("Pilha vazia. Não é possível desempilhar.\n");
        return;
    }
    NoPilha* temp = pilha->topo;
    *valorDesempilhado = temp->valor;
    pilha->topo = temp->prox;
    free(temp);
    pilha->tamanho--;
}
```



PILHAS

Desempilhar: Remover do topo

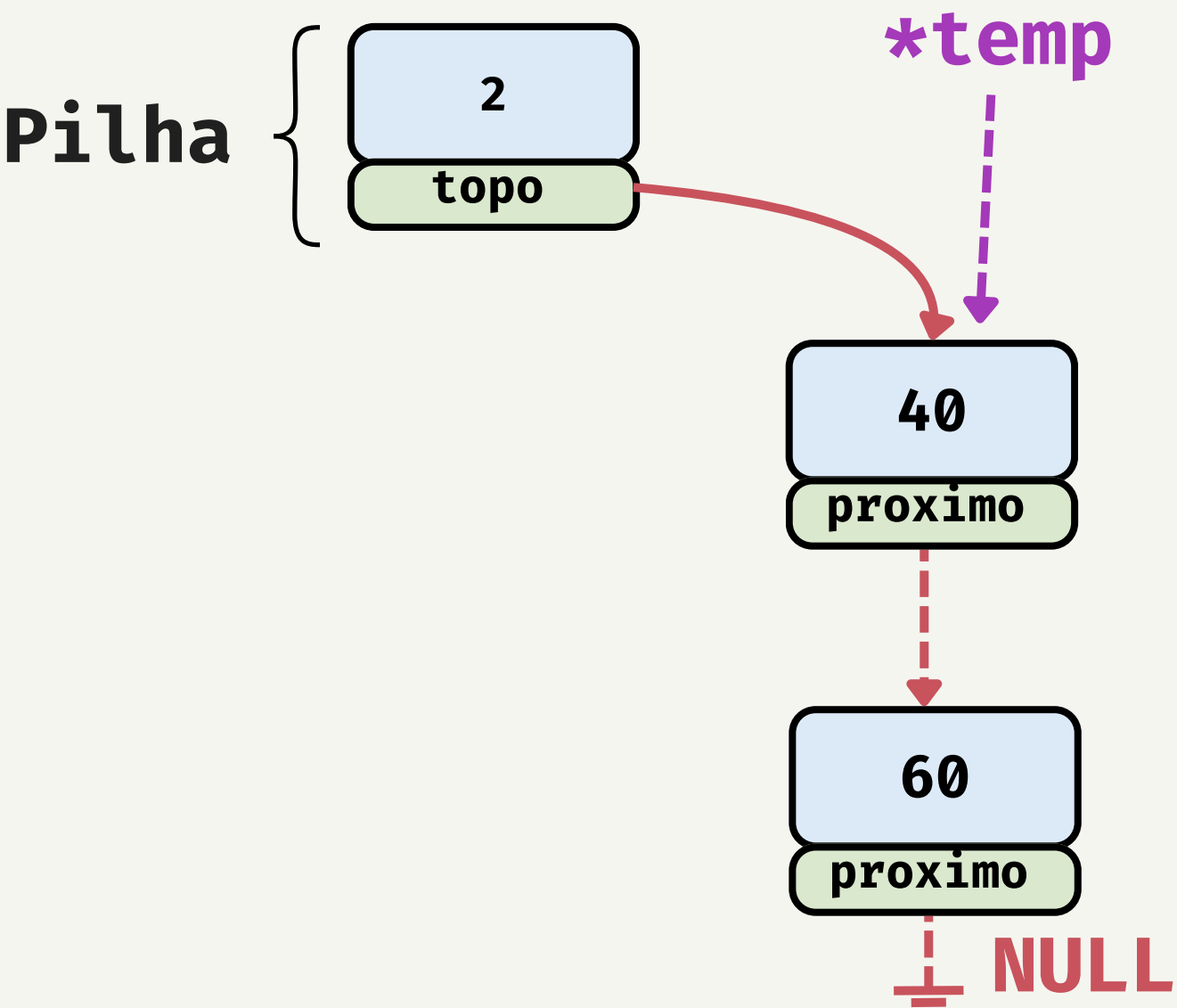
```
void desempilhar(Pilha* pilha, int* valorDesempilhado) {
    if (pilhaVazia(pilha)) {
        printf("Pilha vazia. Não é possível desempilhar.\n");
        return;
    }
    NoPilha* temp = pilha->topo;
    *valorDesempilhado = temp->valor;
    pilha->topo = temp->prox;
    free(temp);
    pilha->tamanho--;
}
```



PILHAS

Desempilhar: Remover do topo

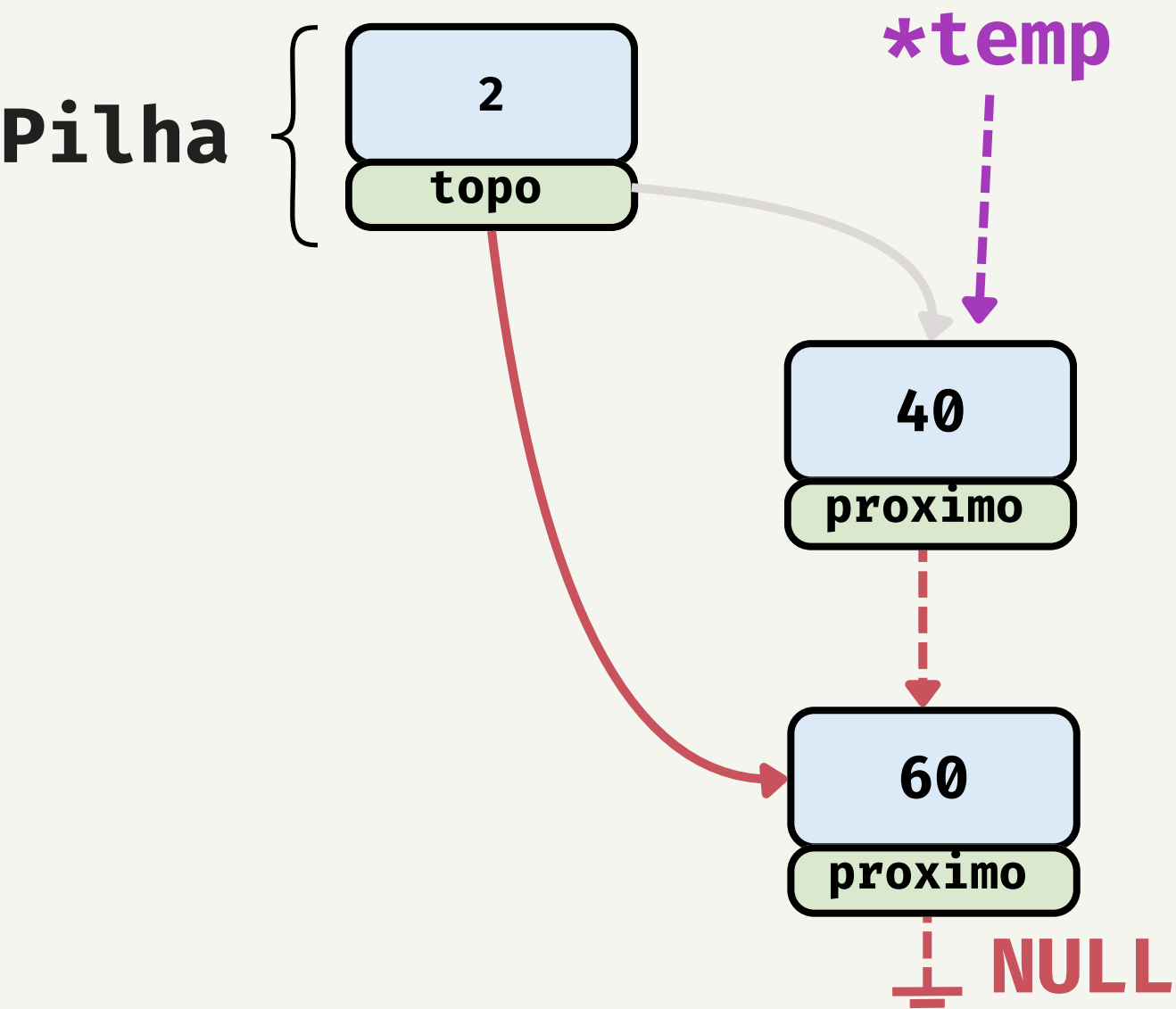
```
void desempilhar(Pilha* pilha, int* valorDesempilhado) {
    if (pilhaVazia(pilha)) {
        printf("Pilha vazia. Não é possível desempilhar.\n");
        return;
    }
    NoPilha* temp = pilha->topo;
    *valorDesempilhado = temp->valor;
    pilha->topo = temp->prox;
    free(temp);
    pilha->tamanho--;
}
```



PILHAS

Desempilhar: Remover do topo

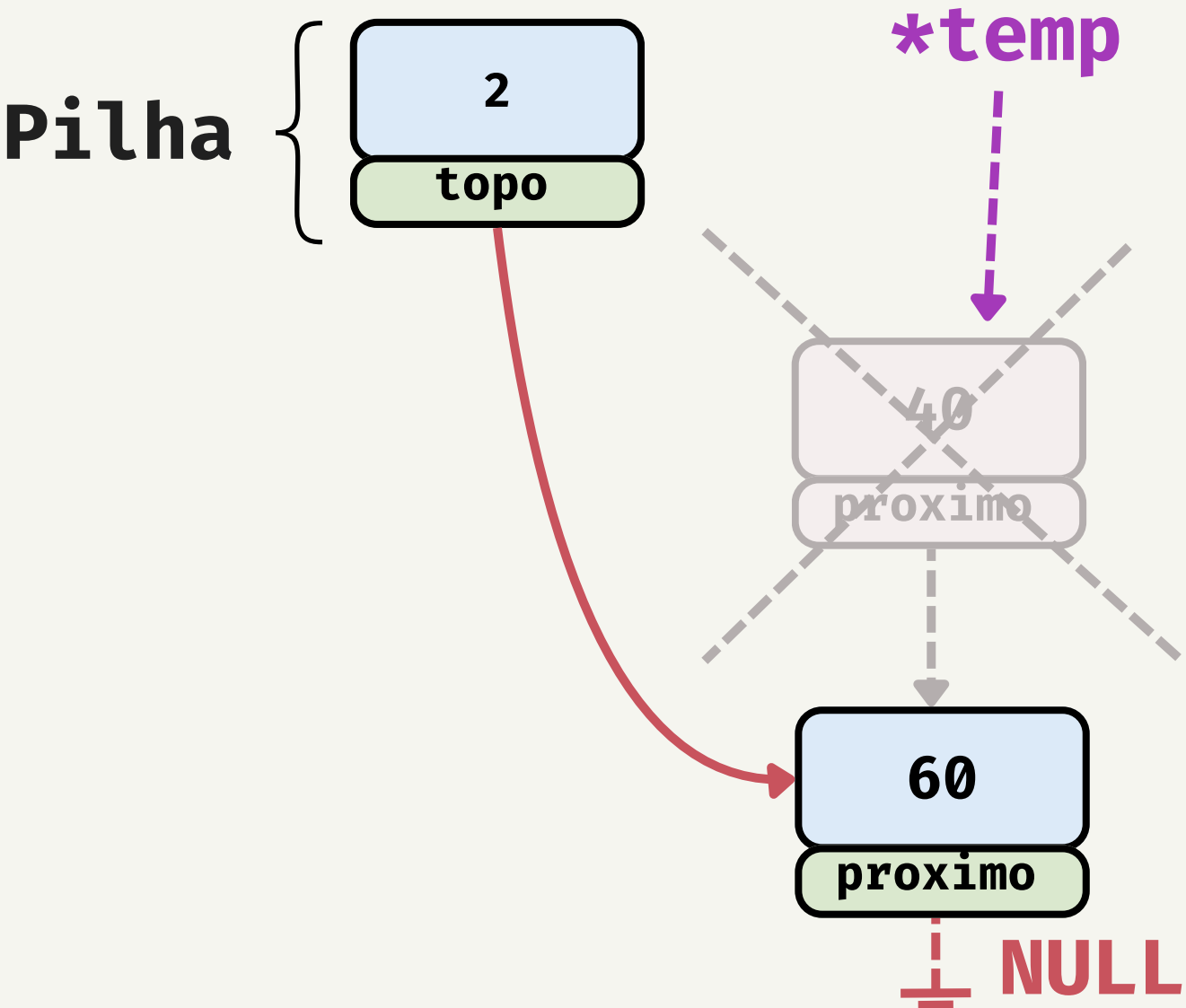
```
void desempilhar(Pilha* pilha, int* valorDesempilhado) {
    if (pilhaVazia(pilha)) {
        printf("Pilha vazia. Não é possível desempilhar.\n");
        return;
    }
    NoPilha* temp = pilha->topo;
    *valorDesempilhado = temp->valor;
    pilha->topo = temp->prox;
    free(temp);
    pilha->tamanho--;
}
```



PILHAS

Desempilhar: Remover do topo

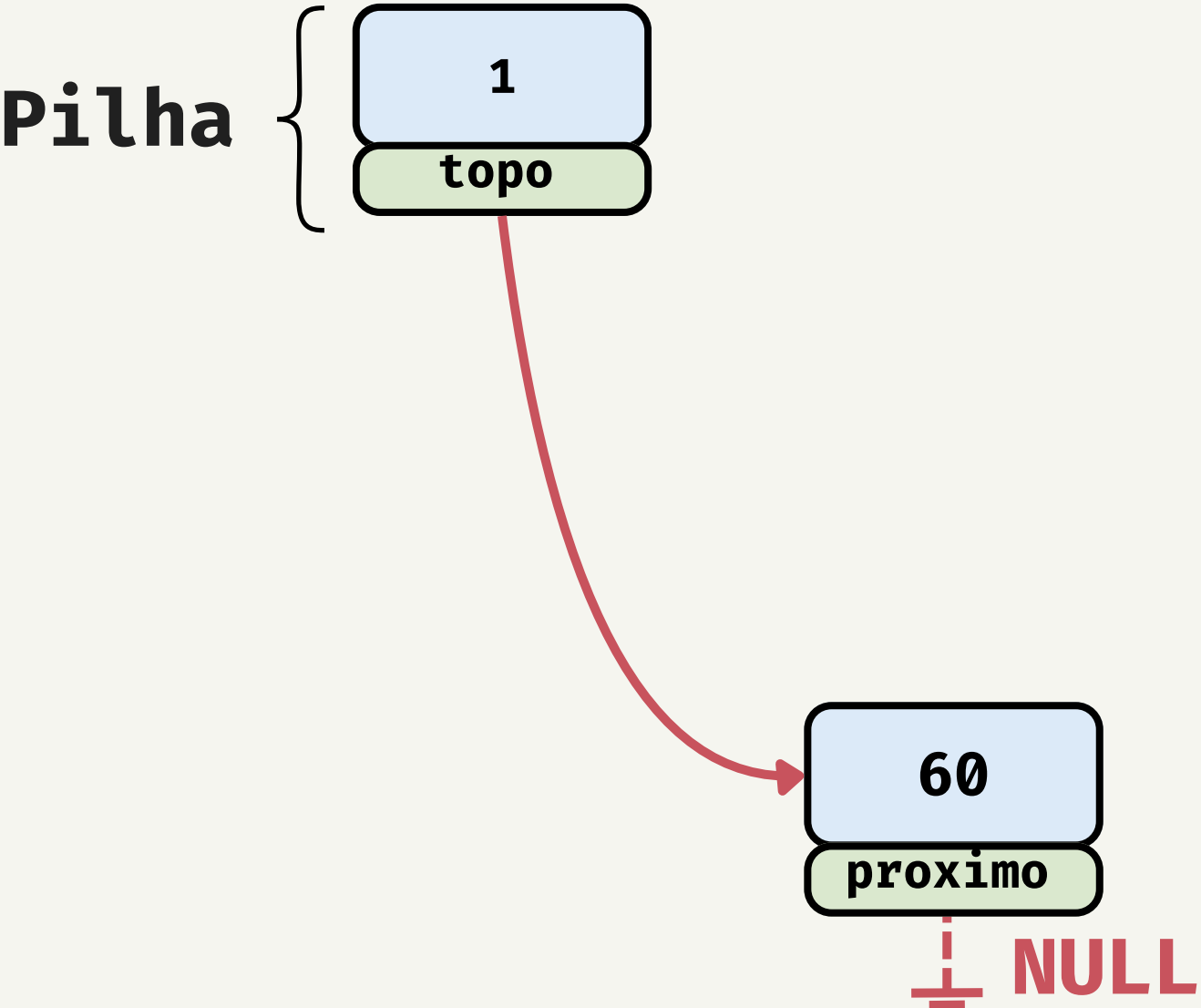
```
void desempilhar(Pilha* pilha, int* valorDesempilhado) {
    if (pilhaVazia(pilha)) {
        printf("Pilha vazia. Não é possível desempilhar.\n");
        return;
    }
    NoPilha* temp = pilha->topo;
    *valorDesempilhado = temp->valor;
    pilha->topo = temp->prox;
    free(temp);
    pilha->tamanho--;
}
```



PILHAS

Desempilhar: Remover do topo

```
void desempilhar(Pilha* pilha, int* valorDesempilhado) {
    if (pilhaVazia(pilha)) {
        printf("Pilha vazia. Não é possível desempilhar.\n");
        return;
    }
    NoPilha* temp = pilha->topo;
    *valorDesempilhado = temp->valor;
    pilha->topo = temp->prox;
    free(temp);
    pilha->tamanho--;
}
```



PILHAS

Apagar pilha: Libera todos os nós e a estrutura pilha

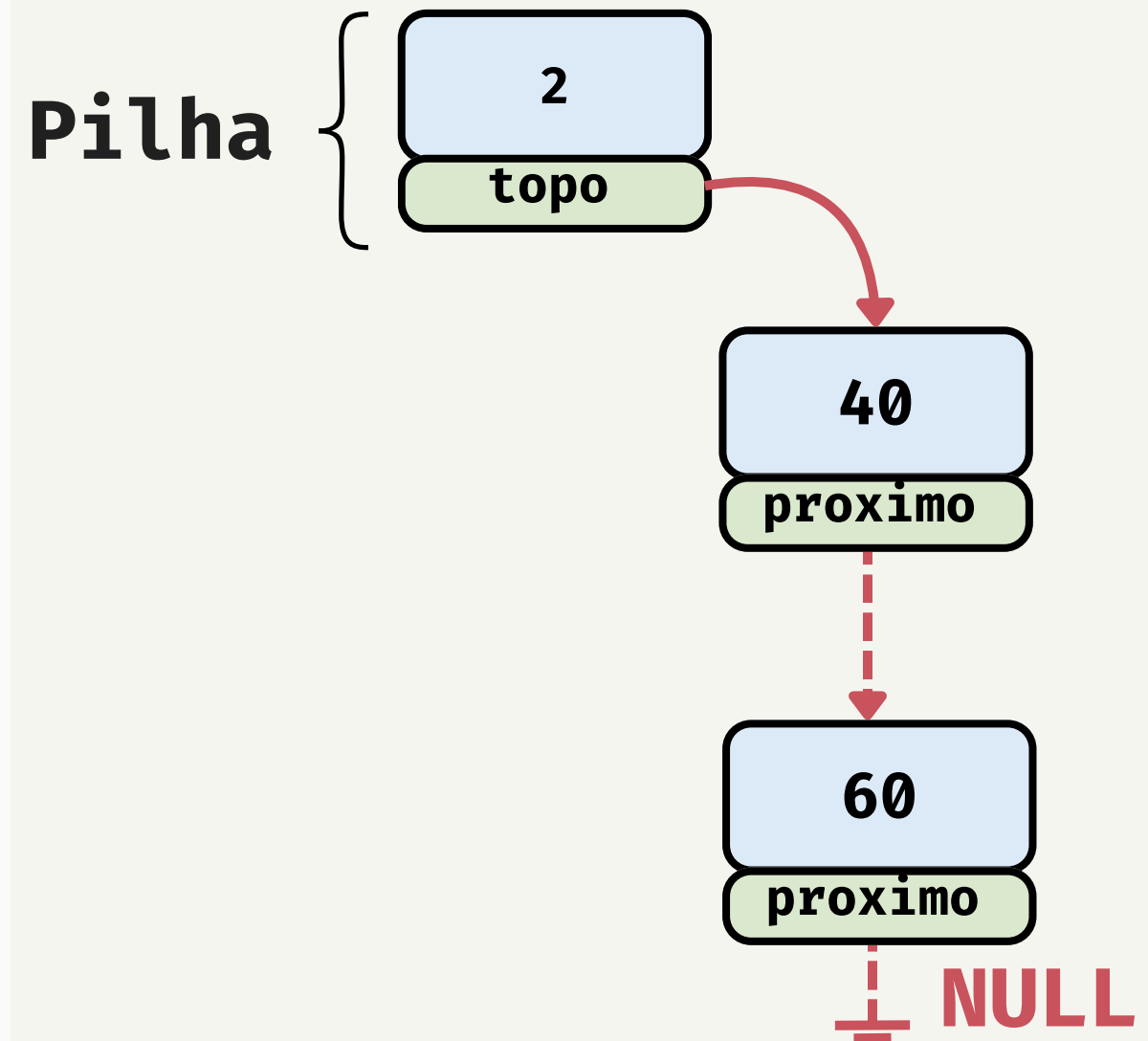
```
void apagarPilha(Pilha* pilha) {  
    if(!pilha) {  
        printf("Pilha nula. Não é possível apagar.\n");  
        return;  
    }  
    NoPilha* atual = pilha->topo;  
    while (atual != NULL) {  
        NoPilha* temp = atual;  
        atual = atual->prox;  
        free(temp);  
    }  
    pilha->topo = NULL;  
    pilha->tamanho = 0;  
    free(pilha);  
}
```

PILHAS

Apagar pilha: Libera todos os nós e a estrutura pilha

```

void apagarPilha(Pilha* pilha) {
    if(!pilha) {
        printf("Pilha nula. Não é possível apagar.\n");
        return;
    }
    NoPilha* atual = pilha->topo;
    while (atual != NULL) {
        NoPilha* temp = atual;
        atual = atual->prox;
        free(temp);
    }
    pilha->topo = NULL;
    pilha->tamanho = 0;
    free(pilha);
}
    
```

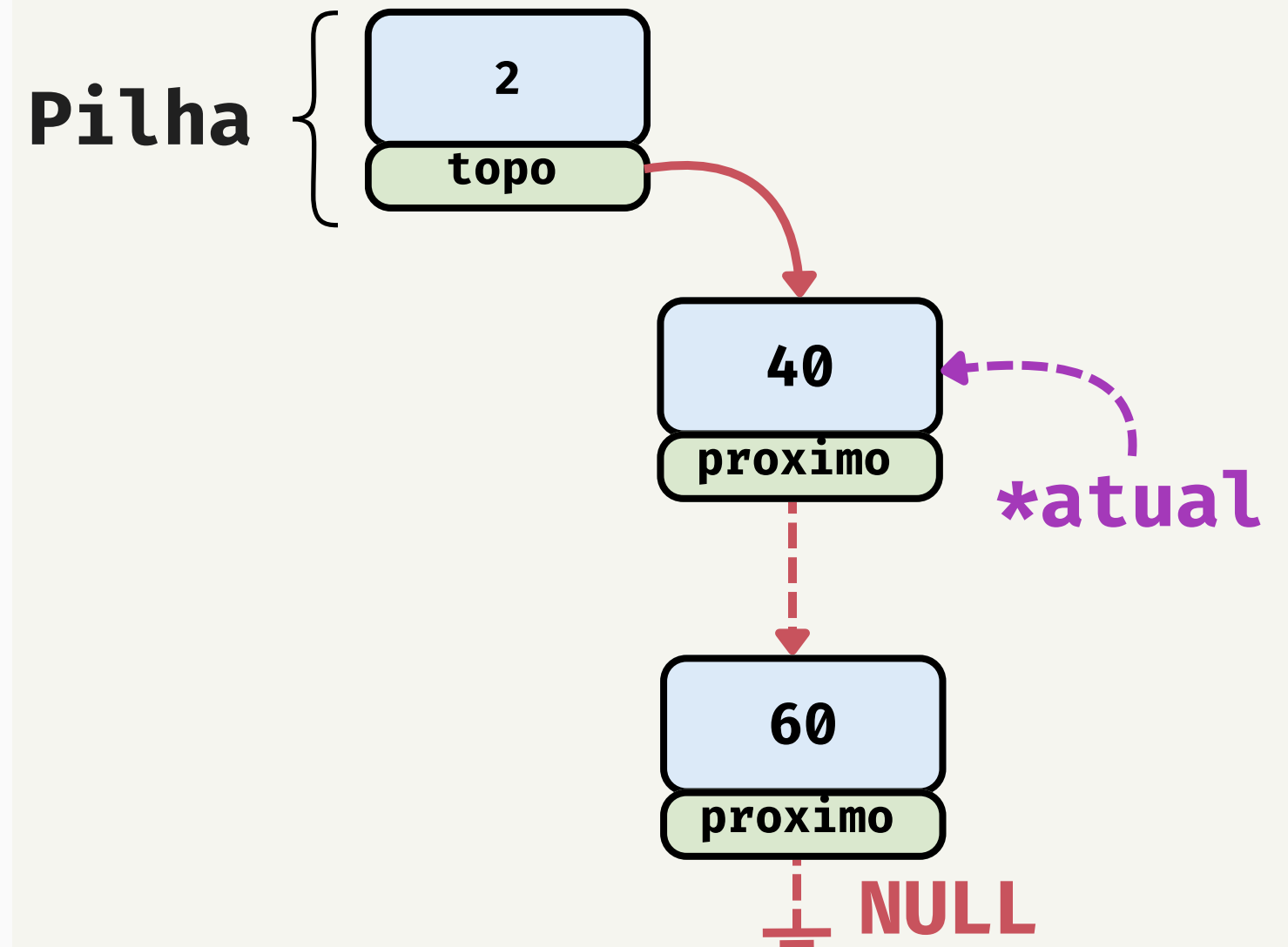


PILHAS

Apagar pilha: Libera todos os nós e a estrutura pilha

```

void apagarPilha(Pilha* pilha) {
    if(!pilha) {
        printf("Pilha nula. Não é possível apagar.\n");
        return;
    }
    NoPilha* atual = pilha->topo;
    while (atual != NULL) {
        NoPilha* temp = atual;
        atual = atual->prox;
        free(temp);
    }
    pilha->topo = NULL;
    pilha->tamanho = 0;
    free(pilha);
}
    
```

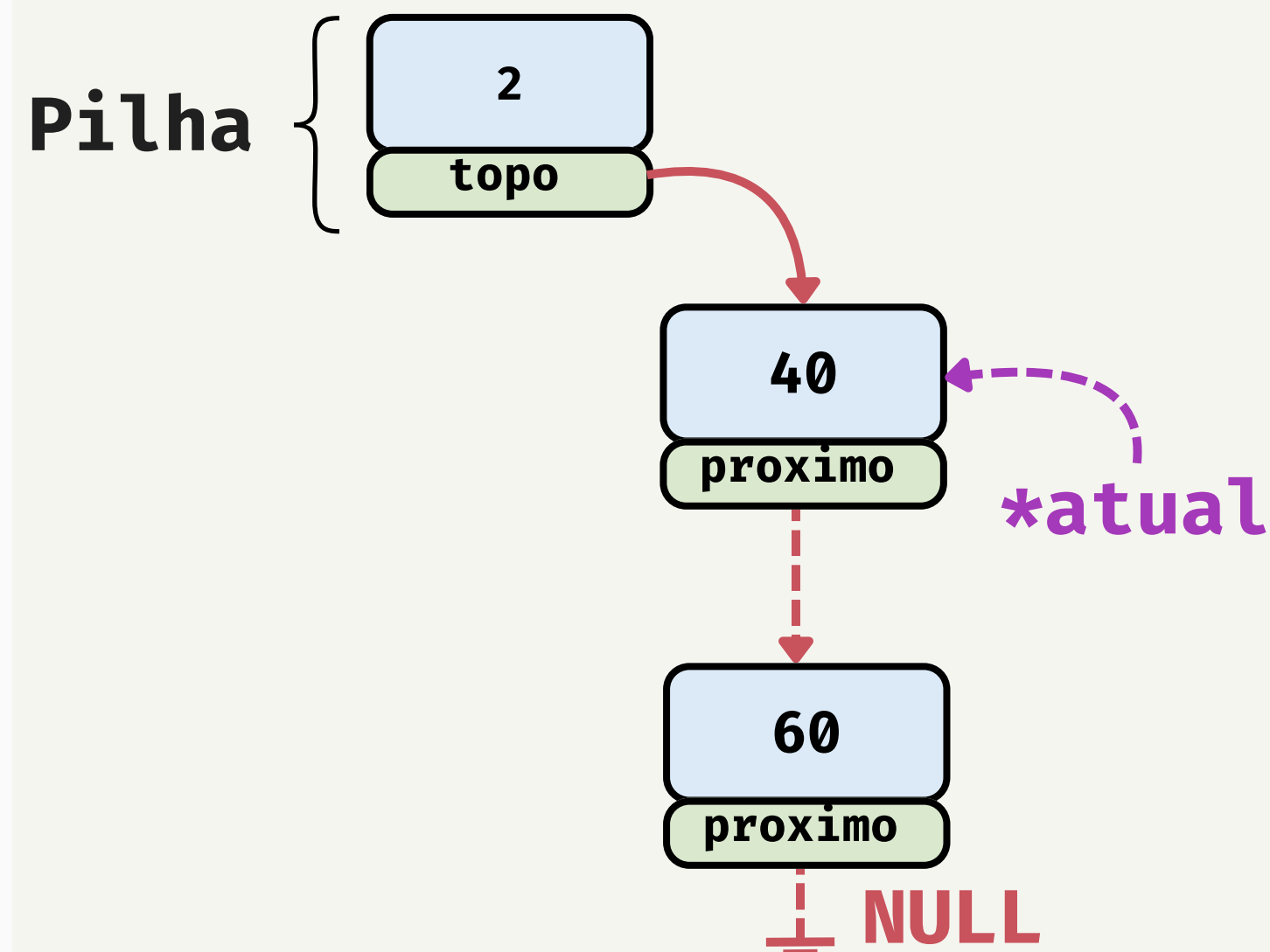


PILHAS

Apagar pilha: Libera todos os nós e a estrutura pilha

```

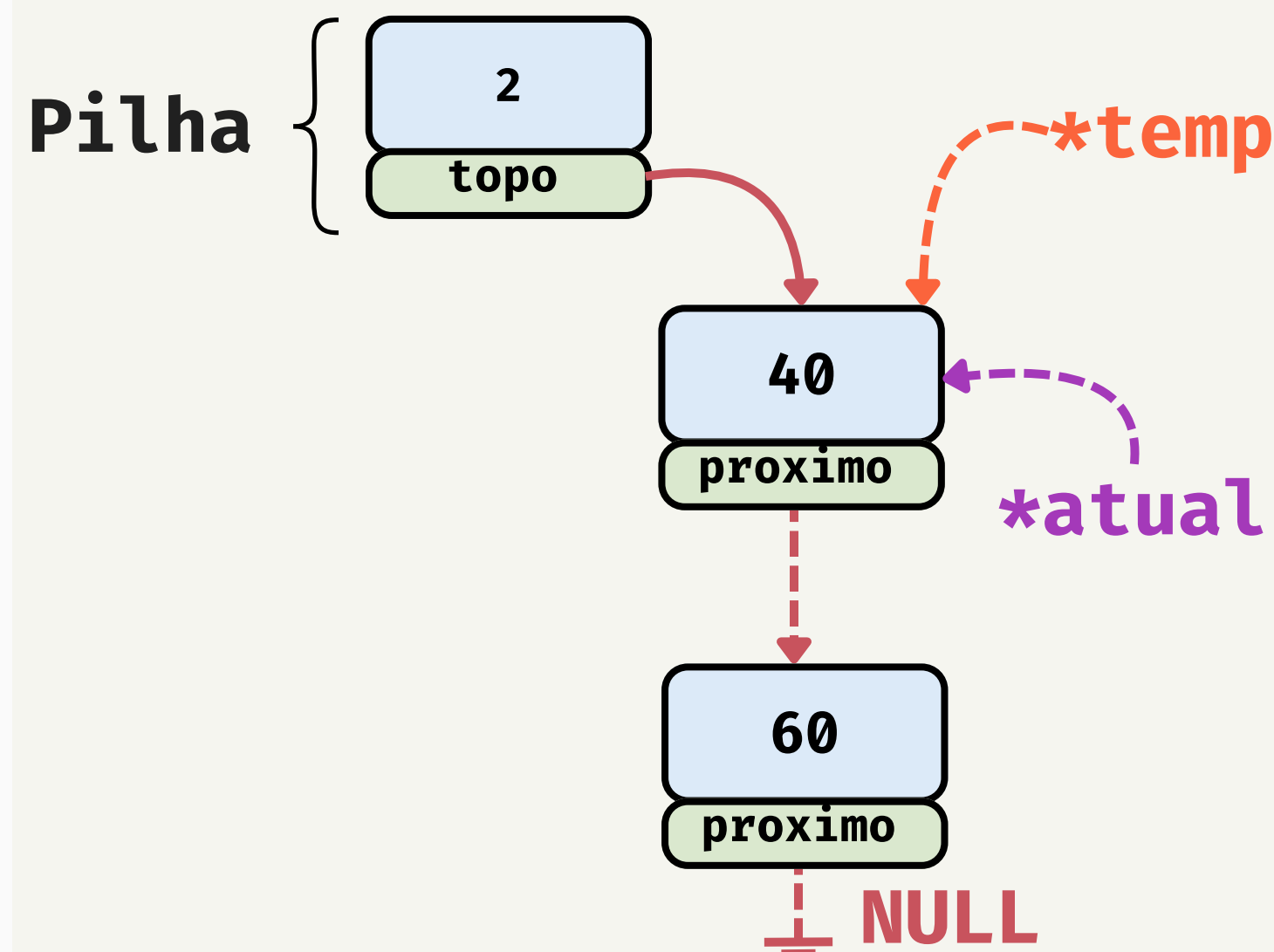
void apagarPilha(Pilha* pilha) {
    if(!pilha) {
        printf("Pilha nula. Não é possível apagar.\n");
        return;
    }
    NoPilha* atual = pilha->topo;
    while (atual != NULL) {
        NoPilha* temp = atual;
        atual = atual->prox;
        free(temp);
    }
    pilha->topo = NULL;
    pilha->tamanho = 0;
    free(pilha);
}
    
```



PILHAS

Apagar pilha: Libera todos os nós e a estrutura pilha

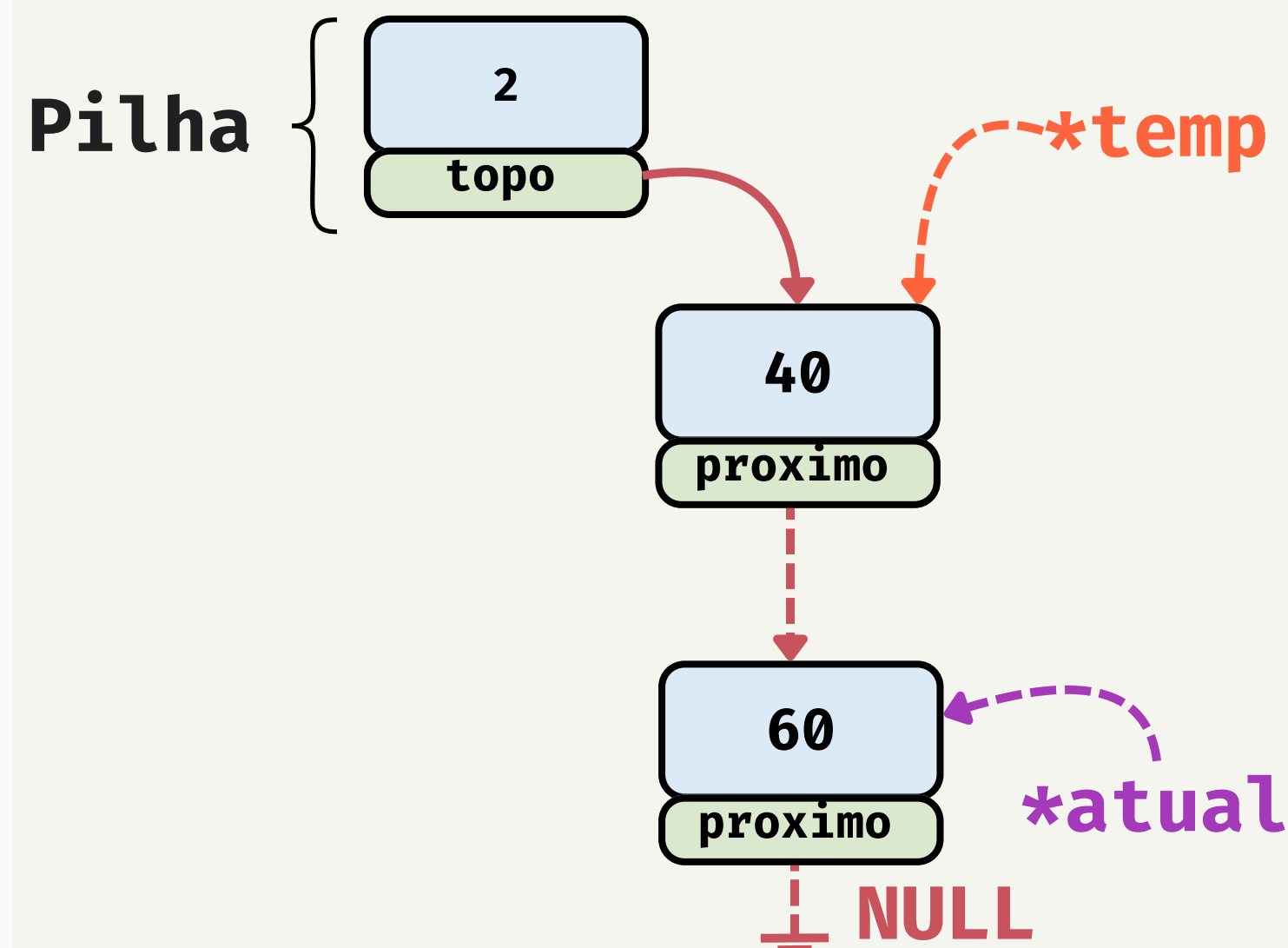
```
void apagarPilha(Pilha* pilha) {
    if(!pilha) {
        printf("Pilha nula. Não é possível apagar.\n");
        return;
    }
    NoPilha* atual = pilha->topo;
    while (atual != NULL) {
        NoPilha* temp = atual;
        atual = atual->prox;
        free(temp);
    }
    pilha->topo = NULL;
    pilha->tamanho = 0;
    free(pilha);
}
```



PILHAS

Apagar pilha: Libera todos os nós e a estrutura pilha

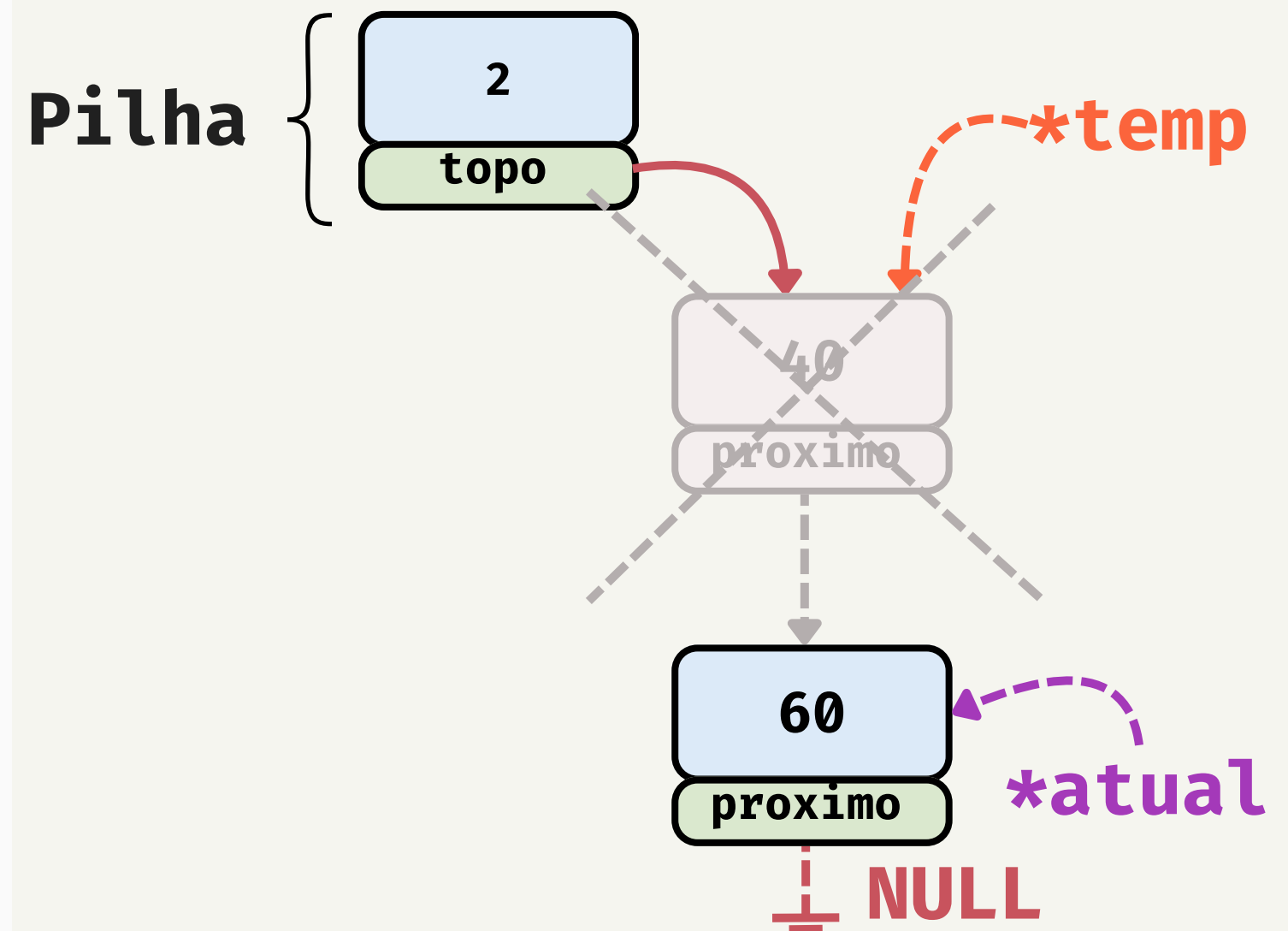
```
void apagarPilha(Pilha* pilha) {
    if(!pilha) {
        printf("Pilha nula. Não é possível apagar.\n");
        return;
    }
    NoPilha* atual = pilha->topo;
    while (atual != NULL) {
        NoPilha* temp = atual;
        atual = atual->prox;
        free(temp);
    }
    pilha->topo = NULL;
    pilha->tamanho = 0;
    free(pilha);
}
```



PILHAS

Apagar pilha: Libera todos os nós e a estrutura pilha

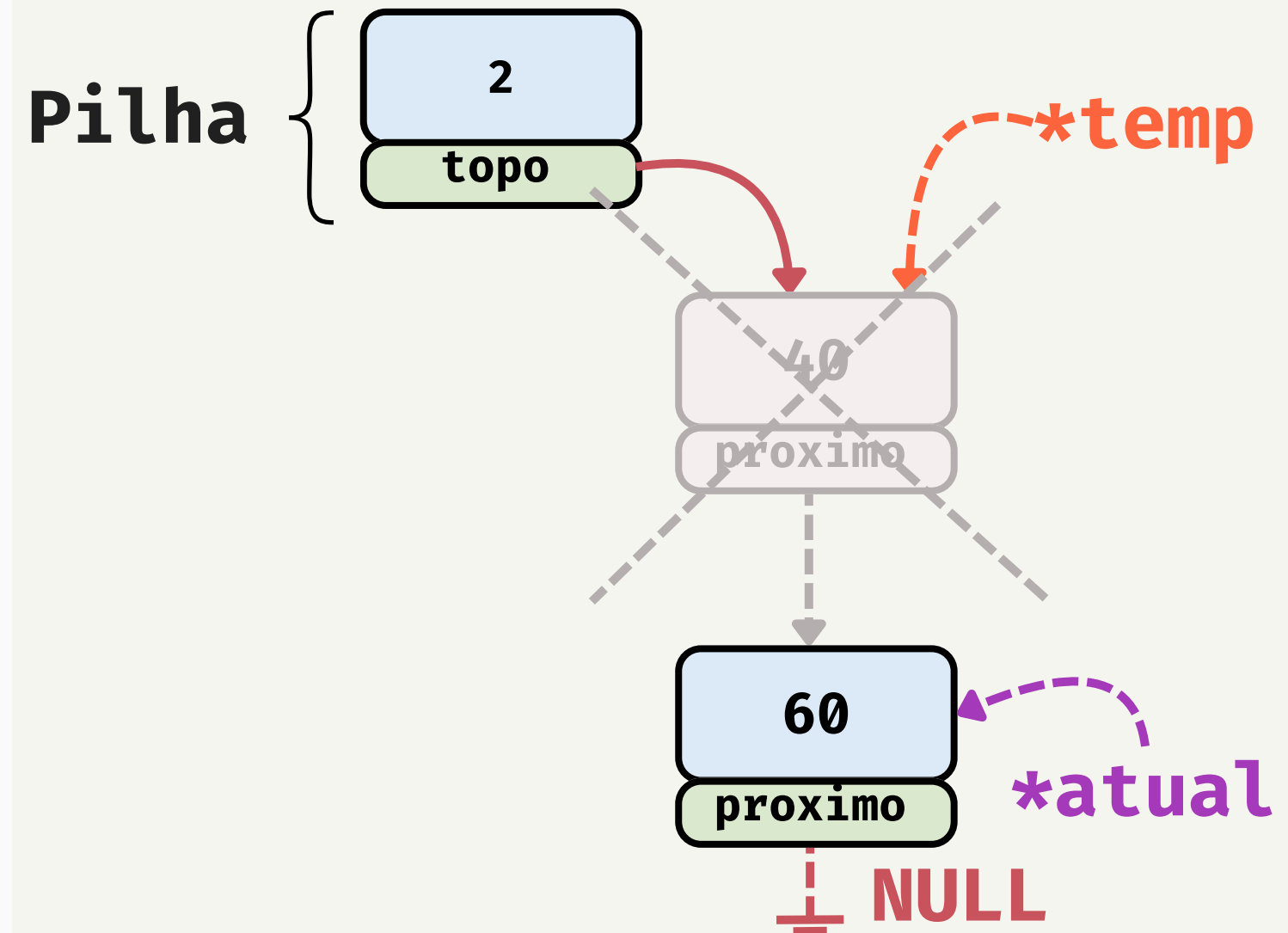
```
void apagarPilha(Pilha* pilha) {
    if(!pilha) {
        printf("Pilha nula. Não é possível apagar.\n");
        return;
    }
    NoPilha* atual = pilha->topo;
    while (atual != NULL) {
        NoPilha* temp = atual;
        atual = atual->prox;
        free(temp);
    }
    pilha->topo = NULL;
    pilha->tamanho = 0;
    free(pilha);
}
```



PILHAS

Apagar pilha: Libera todos os nós e a estrutura pilha

```
void apagarPilha(Pilha* pilha) {
    if(!pilha) {
        printf("Pilha nula. Não é possível apagar.\n");
        return;
    }
    NoPilha* atual = pilha->topo;
    while (atual != NULL) {
        NoPilha* temp = atual;
        atual = atual->prox;
        free(temp);
    }
    pilha->topo = NULL;
    pilha->tamanho = 0;
    free(pilha);
}
```

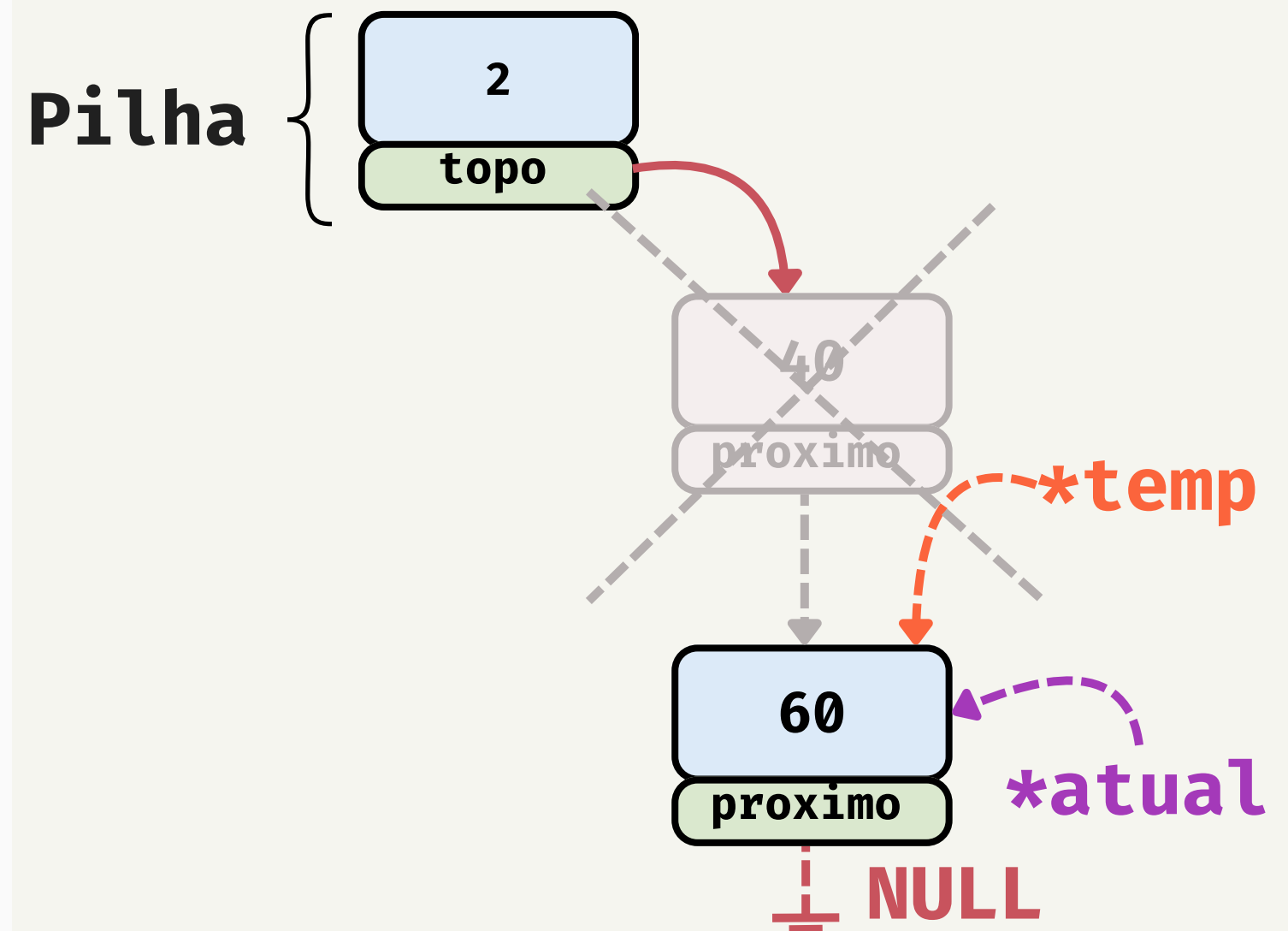


PILHAS

Apagar pilha: Libera todos os nós e a estrutura pilha

```

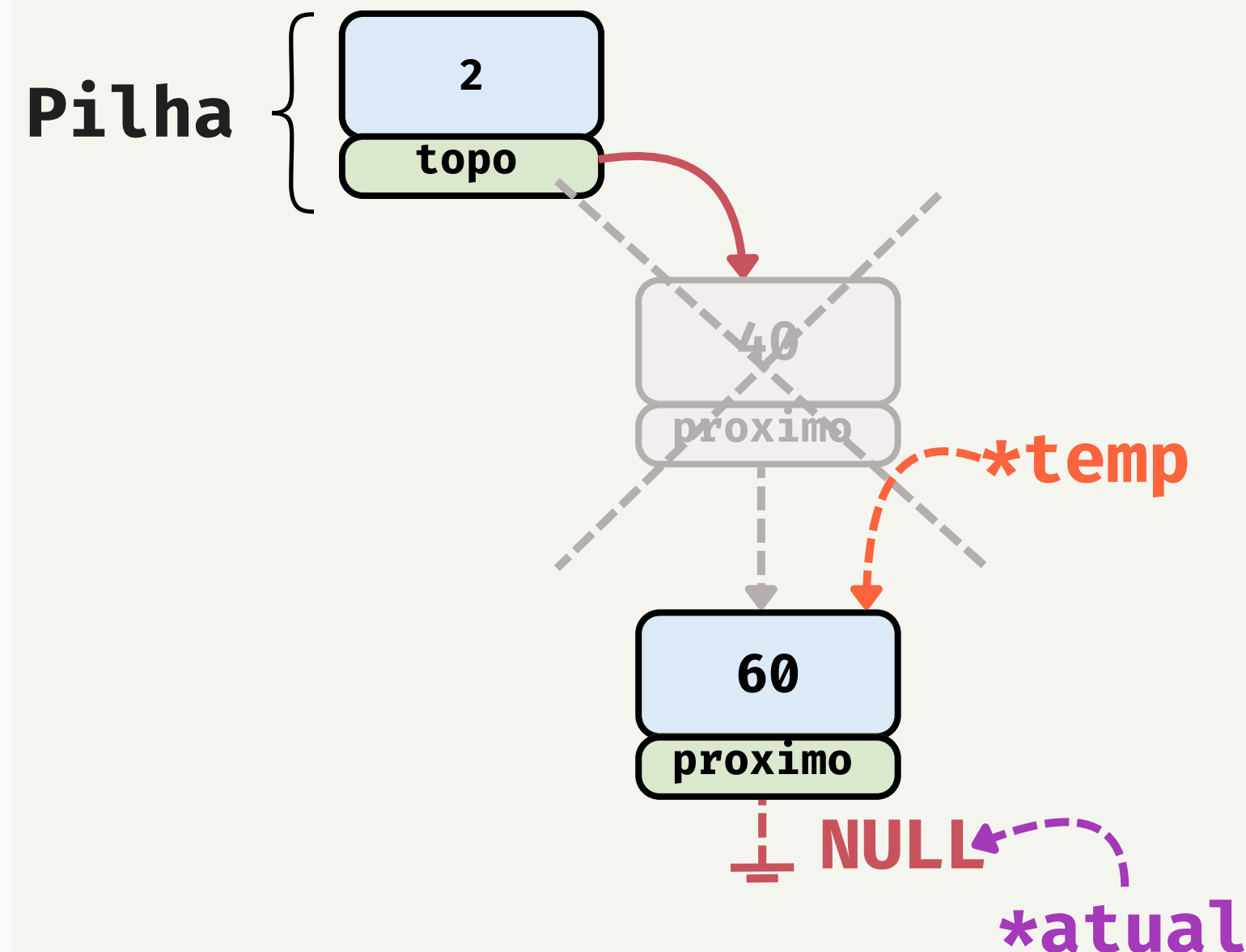
void apagarPilha(Pilha* pilha) {
    if(!pilha) {
        printf("Pilha nula. Não é possível apagar.\n");
        return;
    }
    NoPilha* atual = pilha->topo;
    while (atual != NULL) {
        NoPilha* temp = atual;
        atual = atual->prox;
        free(temp);
    }
    pilha->topo = NULL;
    pilha->tamanho = 0;
    free(pilha);
}
    
```



PILHAS

Apagar pilha: Libera todos os nós e a estrutura pilha

```
void apagarPilha(Pilha* pilha) {
    if(!pilha) {
        printf("Pilha nula. Não é possível apagar.\n");
        return;
    }
    NoPilha* atual = pilha->topo;
    while (atual != NULL) {
        NoPilha* temp = atual;
        atual = atual->prox;
        free(temp);
    }
    pilha->topo = NULL;
    pilha->tamanho = 0;
    free(pilha);
}
```

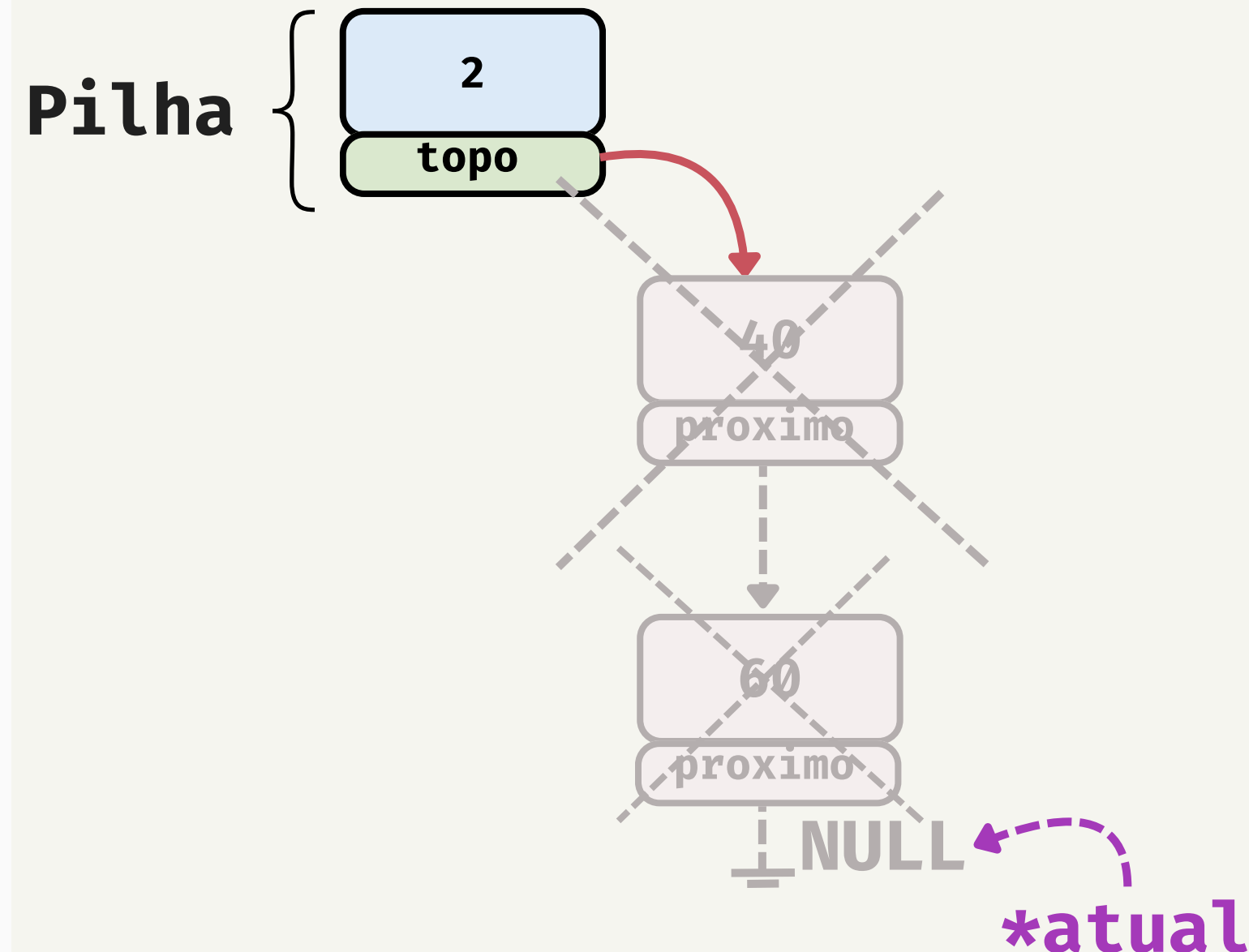


PILHAS

Apagar pilha: Libera todos os nós e a estrutura pilha

```

void apagarPilha(Pilha* pilha) {
    if(!pilha) {
        printf("Pilha nula. Não é possível apagar.\n");
        return;
    }
    NoPilha* atual = pilha->topo;
    while (atual != NULL) {
        NoPilha* temp = atual;
        atual = atual->prox;
        free(temp);
    }
    pilha->topo = NULL;
    pilha->tamanho = 0;
    free(pilha);
}
    
```

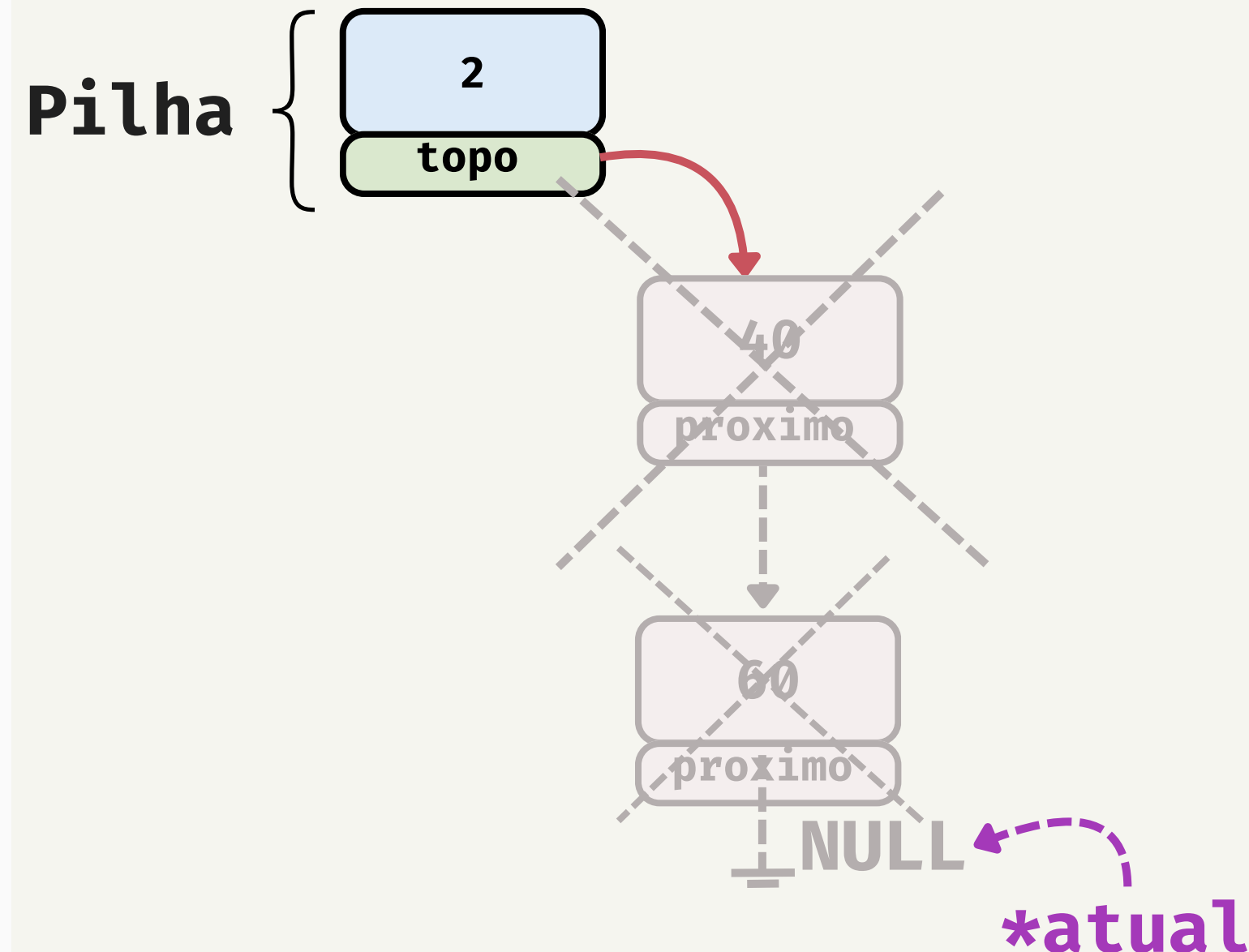


PILHAS

Apagar pilha: Libera todos os nós e a estrutura pilha

```

void apagarPilha(Pilha* pilha) {
    if(!pilha) {
        printf("Pilha nula. Não é possível apagar.\n");
        return;
    }
    NoPilha* atual = pilha->topo;
    while (atual != NULL) {
        NoPilha* temp = atual;
        atual = atual->prox;
        free(temp);
    }
    pilha->topo = NULL;
    pilha->tamanho = 0;
    free(pilha);
}
    
```

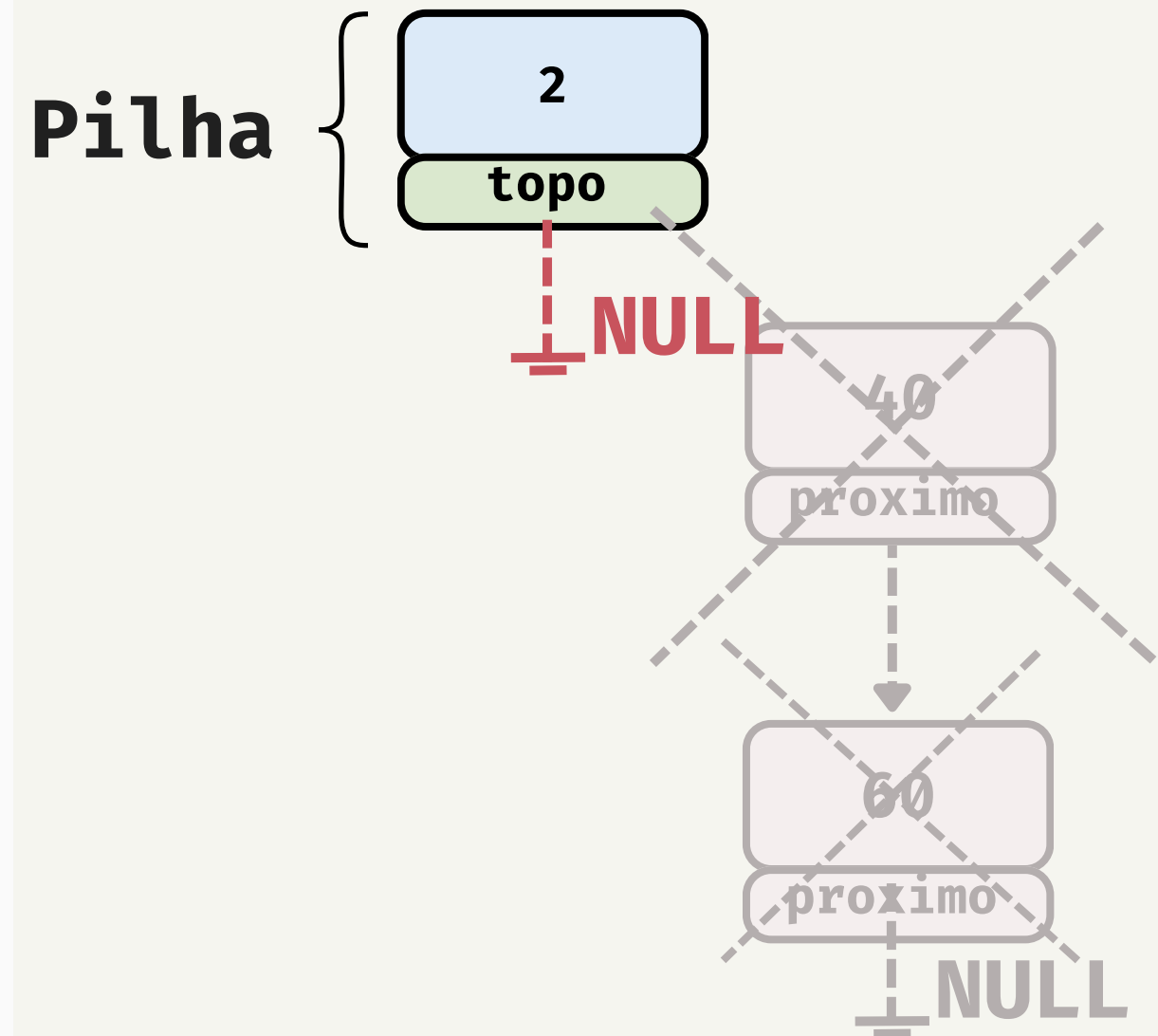


PILHAS

Apagar pilha: Libera todos os nós e a estrutura pilha

```

void apagarPilha(Pilha* pilha) {
    if(!pilha) {
        printf("Pilha nula. Não é possível apagar.\n");
        return;
    }
    NoPilha* atual = pilha->topo;
    while (atual != NULL) {
        NoPilha* temp = atual;
        atual = atual->prox;
        free(temp);
    }
    pilha->topo = NULL;
    pilha->tamanho = 0;
    free(pilha);
}
    
```

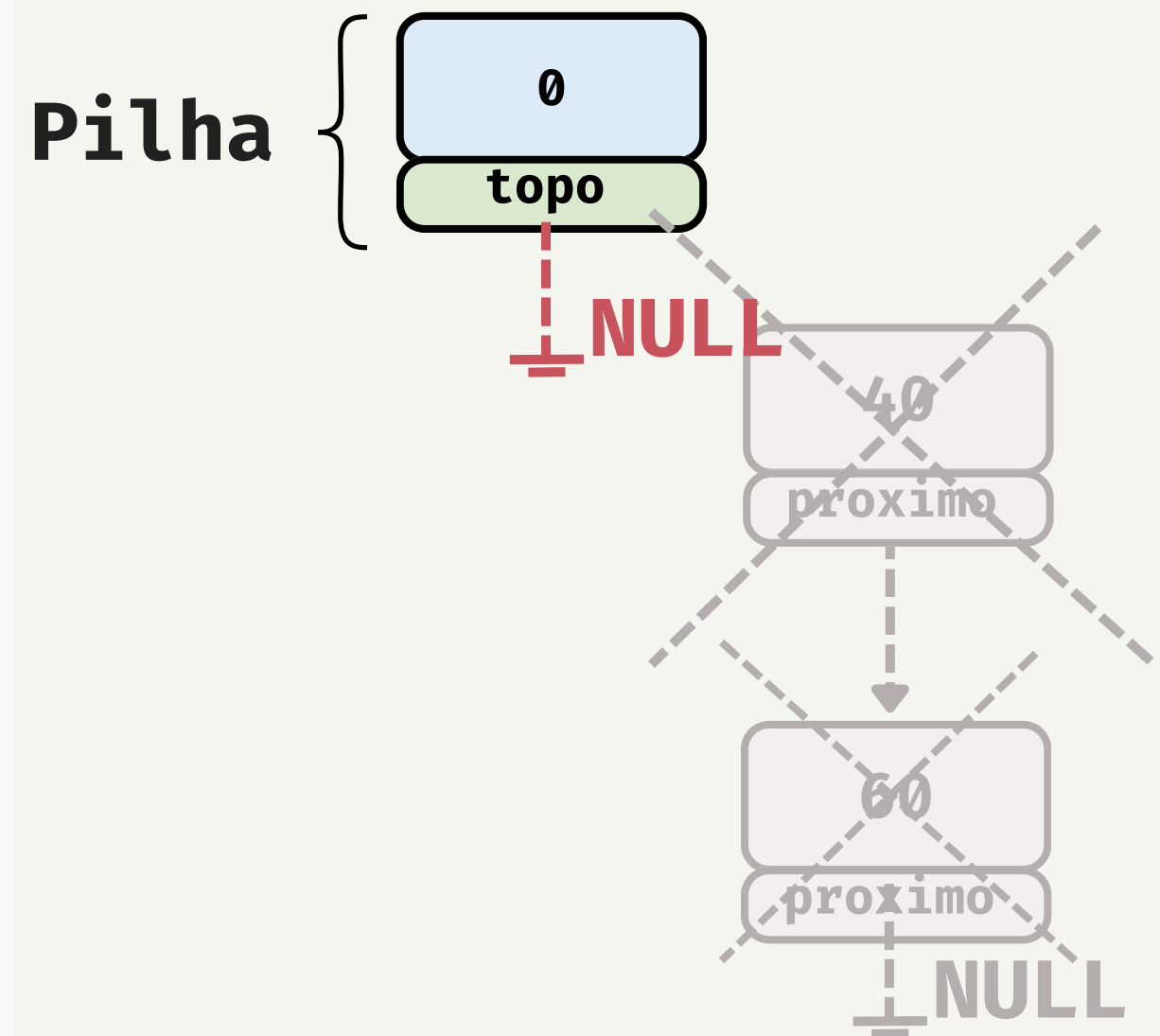


PILHAS

Apagar pilha: Libera todos os nós e a estrutura pilha

```

void apagarPilha(Pilha* pilha) {
    if(!pilha) {
        printf("Pilha nula. Não é possível apagar.\n");
        return;
    }
    NoPilha* atual = pilha->topo;
    while (atual != NULL) {
        NoPilha* temp = atual;
        atual = atual->prox;
        free(temp);
    }
    pilha->topo = NULL;
    pilha->tamanho = 0;
    free(pilha);
}
    
```

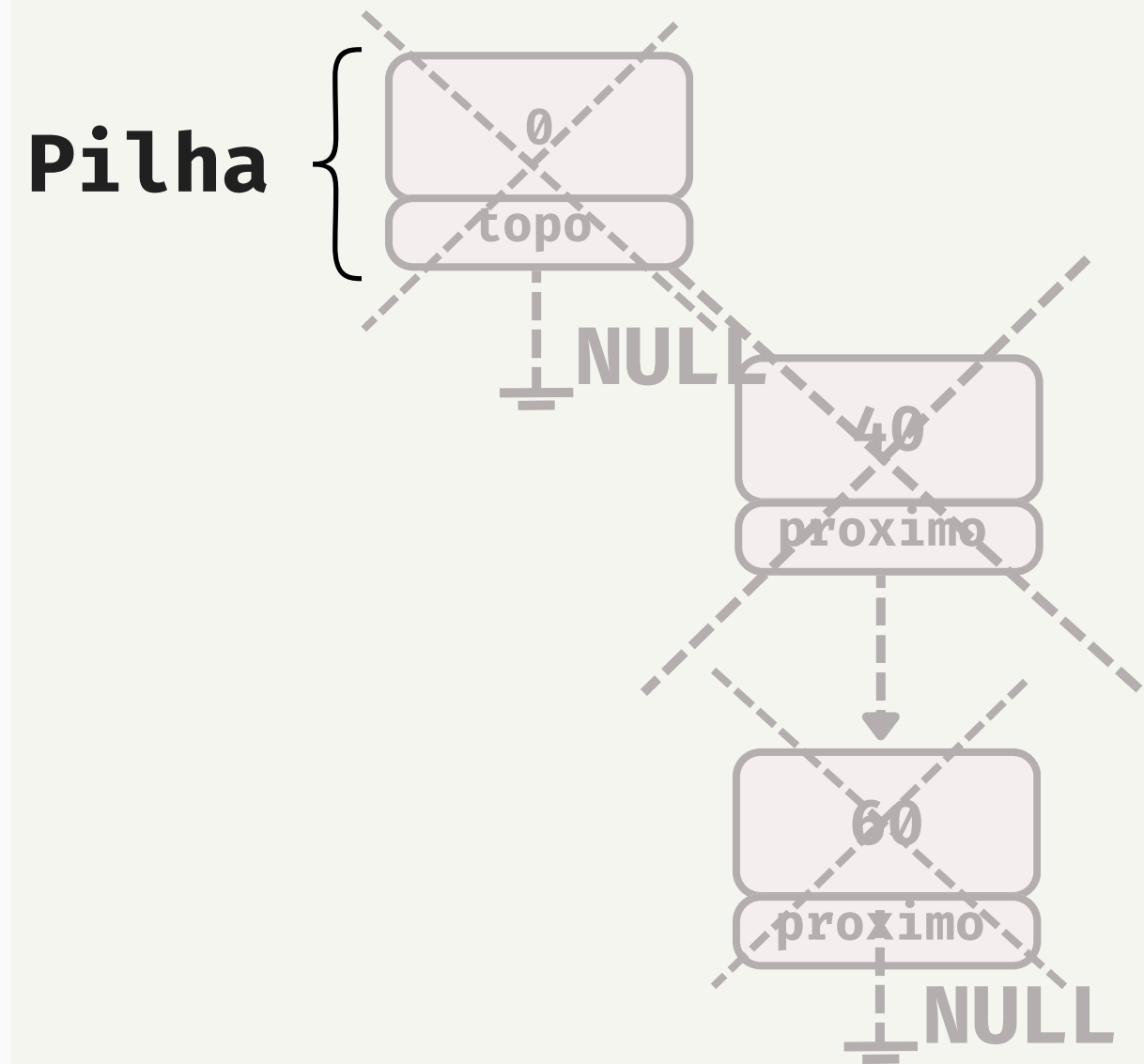


PILHAS

Apagar pilha: Libera todos os nós e a estrutura pilha

```

void apagarPilha(Pilha* pilha) {
    if(!pilha) {
        printf("Pilha nula. Não é possível apagar.\n");
        return;
    }
    NoPilha* atual = pilha->topo;
    while (atual != NULL) {
        NoPilha* temp = atual;
        atual = atual->prox;
        free(temp);
    }
    pilha->topo = NULL;
    pilha->tamanho = 0;
    free(pilha);
}
    
```



PILHAS

Outras funções importantes

OBS: const → garante que a função **não altera** a variável

Retorna **1** se a pilha estiver **vazia**, **0** caso **contrário**

```
int pilhaVazia(const Pilha* pilha) {  
    return (pilha->tamanho == 0);  
}
```

PILHAS

Outras funções importantes

OBS: const → garante que a função **não altera** a variável

Retorna **1** se a pilha estiver **vazia**, **0** caso **contrário**

```
int pilhaVazia(const Pilha* pilha) {  
    return (pilha->tamanho == 0);  
}
```

Retorna o **valor** que está no topo da pilha

```
int topoPilha(const Pilha* pilha) {  
    if (pilhaVazia(pilha)) {  
        printf("Pilha vazia. Não há topo para retornar.\n");  
        return -1;  
    }  
    return pilha->topo->valor;  
}
```

PILHAS

Outras funções importantes

Atualiza o **valor** do nó no **topo** da pilha

```
void atualizarTopo(Pilha* pilha, int novoValor) {  
    if (pilhaVazia(pilha)) {  
        printf("Pilha vazia. Não é possível atualizar o topo.\n");  
        return;  
    }  
    pilha->topo->valor = novoValor;  
}
```


PILHAS

Outras funções importantes

Imprime a pilha

```
void imprimirPilha(const Pilha* pilha) {  
    NoPilha* atual = pilha->topo;  
    printf("Pilha (topo para base): ");  
    while (atual != NULL) {  
        printf("\n%d ", atual->valor);  
        atual = atual->prox;  
    }  
    printf("\nNULL\n");  
}
```

PILHAS COMO TAD

Quando falamos em **Pilha**, pensamos em:

Operações:

empilhar()/push()

desempilhar()/pop()

pilhaVazia()

Regra de funcionamento:

📌 **LIFO** (Last In, First Out)

O **usuário** não precisa saber que ela é implementada com:

- alocação dinâmica de memória
- Não sabe se existe topo
- Não sabe se é vetor ou lista

PILHAS COMO TAD

uso do **STATIC**

Uma função **static** só pode ser chamada dentro de um **mesmo** arquivo que foi criada (**definida**)

- Evita conflitos de nomes com funções de **mesmo nome** em outros arquivos do projeto

PILHAS COMO TAD

uso do **STATIC**

Uma função **static** só pode ser chamada dentro de um **mesmo** arquivo que foi criada (**definida**)

- Evita conflitos de nomes com funções de **mesmo nome** em outros arquivos do projeto

```
static NoPilha* criarNo(int valor) {
    NoPilha* novoNo = (NoPilha*)malloc(sizeof(NoPilha));
    if (!novoNo) {
        printf("Erro ao alocar memória para o nó da pilha.\n");
    }
    novoNo->valor = valor;
    novoNo->prox = NULL;
    return novoNo;
}
```

```
void empilhar(Pilha* pilha, int valor) {
    NoPilha* novoNo = criarNo(valor);
    novoNo->prox = pilha->topo;
    pilha->topo = novoNo;
    pilha->tamanho++;
}
```

PILHAS

Exercício: Inverter a ordem dos números digitados

PILHAS

Exercício: Inverter a ordem dos números digitados

O que acontece com a ordem dos elementos ao desempilhar?

PILHAS

Exercício: Inverter a ordem dos números digitados

O que acontece com a ordem dos elementos ao desempilhar?

Ler 5 números inteiros.

Empilhe cada número digitado.

Desempilhe todos os elementos e imprima na tela.

PILHAS COMO TAD

Exercício: Inverter a ordem dos números digitados usando pilhas

```
int main() {
    Pilha* pilha = criarPilha();

    if (!pilha) {
        printf("Erro ao criar pilha.\n");
        return 1;
    }

    int numero;

    printf("Digite 5 numeros inteiros:\n");

    for (int i = 0; i < 5; i++) {
        scanf("%d", &numero);
        empilhar(pilha, numero);
    }
}
```

```
printf("\nNumeros na ordem inversa:\n");

while (!pilhaVazia(pilha)) {
    desempilhar(pilha, &numero);
    printf("%d\n", numero);
}

apagarPilha(pilha);

return 0;
}
```


PILHAS COMO TAD

Exercício: Verifique se uma palavra é um **palíndromo**

PILHAS COMO TAD

Exercício: Verifique se uma palavra é um **palíndromo**

Palíndromo: palavra é igual quando lida de trás para frente

PILHAS COMO TAD

Exercício: Verifique se uma palavra é um **palíndromo**

Palíndromo: palavra é igual quando lida de trás para frente

1. **Percorra** a palavra da esquerda para a direita.
2. **Empilhe** cada letra.
3. Depois **percorra** novamente a palavra:
4. **desempilhe** uma letra
5. **compare** com a letra da posição atual
6. **Se** alguma for diferente → não é palíndromo.

⚠ Em C:

char pode ser convertido automaticamente para **int**

PILHAS COMO TAD

Exercício: Verifique se uma palavra é um **palíndromo**

```
int main() {
    Pilha* pilha = criarPilha();
    if (!pilha) return 1;

    char palavra[100];
    int i, valor;
    int ehPalindromo = 1;

    printf("Digite uma palavra: ");
    scanf("%s", palavra);

    int tamanho = strlen(palavra);

    // Empilha cada caractere
    for (i = 0; i < tamanho; i++) {
        empilhar(pilha, palavra[i]);
    }
```

```
// Compara desempilhando
for (i = 0; i < tamanho; i++) {
    desempilhar(pilha, &valor);

    if (palavra[i] != (char)valor) {
        ehPalindromo = 0;
        break;
    }
}

if (ehPalindromo)
    printf("Eh palindromo!\n");
else
    printf("Nao eh palindromo!\n");

apagarPilha(pilha);
return 0;
}
```

PILHAS - EXERCÍCIOS

Ctrl+Z / Ctrl+Y (Desfazer e Refazer)

Vamos simular um “**editor**” que tem um **estado numérico** (por exemplo: tamanho do texto, pontuação, saldo etc.).

Teremos **duas pilhas**:

- 1. desfazer** (undo): guarda estados **anteriores**
- 2. refazer** (redo): guarda estados que foram **desfeitos**

PILHAS - EXERCÍCIOS

Ctrl+Z / Ctrl+Y (Desfazer e Refazer) usando Pilha

O programa deve **ler** os seguintes comandos:

- **D x** → (Digitar) aplica uma mudança: estado += x
- **Z** → (Desfazer / Ctrl+Z)
- **Y** → (Refazer / Ctrl+Y)
- **P** → imprime o estado

PILHAS - EXERCÍCIOS

Ctrl+Z / Ctrl+Y (Desfazer e Refazer) usando Pilha

Regras:

Ao executar D x:

1. **Empilhar** o estado atual na pilha desfazer
2. **Atualizar**: estado = estado + x
3. **Esvaziar** a pilha refazer (porque uma ação nova **invalida** o refazer)

Ao executar Z:

Se a pilha desfazer estiver **vazia** → não faz nada

Senão:

1. **Empilhar** o estado atual na pilha **refazer**
2. **Desempilhar** da pilha **desfazer** e atribuir ao estado

OBS: O REFAZER EXISTE SE EU DESFIZ ALGO. AO EXECUTAR UMA NOVA AÇÃO, UMA NOVA LINHA DO TEMPO É EXECUTADA E NÃO EXISTE MAIS REFAZER.

Ao executar Y:

Se a pilha refazer estiver **vazia** → não faz nada

Senão:

1. **Empilhar** o estado atual na pilha **desfazer**
2. **Desempilhar** da pilha **refazer** e atribuir ao estado

PILHAS - EXERCÍCIOS

Ctrl+Z / Ctrl+Y (Desfazer e Refazer) usando Pilha

Funções que já construímos

- **empilhar(pilha, valor)**
- **desempilhar(pilha, &valor)**
- **pilhaVazia(pilha)**
- **imprimirPilha(pilha)**

Necessário construir uma função para esvaziar a pilha sem dar free(pilha)

→ preciso reutilizar a pilha **refazer** após esvaziar.

Função nova: **esvaziarPilha(Pilha* pilha)**

PILHAS - EXERCÍCIOS

Ctrl+Z / Ctrl+Y (Desfazer e Refazer) usando Pilha

Precisamos construir uma função para esvaziar a pilha sem dar free(pilha)

→ preciso reutilizar a pilha após esvaziar.

esvaziarPilha(Pilha* pilha)

```

void apagarPilha(Pilha* pilha) {
    if(!pilha) {
        printf("Pilha nula. Não é possível apagar.\n");
        return;
    }
    NoPilha* atual = pilha->topo;
    while (atual != NULL) {
        NoPilha* temp = atual;
        atual = atual->prox;
        free(temp);
    }
    pilha->topo = NULL;
    pilha->tamanho = 0;
    free(pilha);
}
    
```

PILHAS - EXERCÍCIOS

Ctrl+Z / Ctrl+Y (Desfazer e Refazer) usando Pilha

```
se comando == "D x":
    empilhar(desfazer, estado)
    estado += x
    esvaziar(refazer)

se comando == "Z":
    se desfazer não vazia:
        empilhar(refazer, estado)
        desempilhar(desfazer, &estado)

se comando == "Y":
    se refazer não vazia:
        empilhar(desfazer, estado)
        desempilhar(refazer, &estado)

se comando == "P":
    print(estado)
```